*Last updated on **May 8, 2026***

Lakebase Postgres

Lakebase Postgres is a fully managed, cloud-native PostgreSQL database that brings online transaction processing (OLTP) capabilities to the Lakehouse.

INFO

Lakebase Autoscaling is the latest version of Lakebase, with autoscaling compute, scale-to-zero, branching, and instant restore. For supported regions, see Region availability. If you are a Lakebase Provisioned user, see Lakebase Provisioned.

INFO

Lakebase Provisioned is the original Lakebase offering that uses provisioned compute you scale manually. For supported regions, see Region availability. For the latest version of Lakebase, with autoscaling compute, scale-to-zero, branching, and instant restore, see Lakebase Autoscaling.

New Lakebase instances will be created as Autoscaling projects. Rollout starts March 12, 2026. For details, see Autoscaling by default.

Lakebase Autoscaling for new databases

New Lakebase instances are created as Lakebase Autoscaling projects. Existing Provisioned instances continue to be supported. For details on the change and what it means for you, see Autoscaling by default.

Version	Description
Lakebase Autoscaling	**Default for new Lakebase instances.** Fully managed PostgreSQL with autoscaling compute, branching, scale-to-zero, and instant restore. Organizes resources as *projects*. See What is Lakebase Autoscaling?.
Lakebase Provisioned	**Existing Provisioned instances only.** We continue to support existing Provisioned instances. Organizes resources as *database instances* with manually scaled compute. See What is Lakebase Provisioned?.

What's new in Lakebase Autoscaling

Lakebase Autoscaling introduces modern database capabilities designed for flexible development and cost-efficient operations:

- **Lakebase App:** A new Lakebase interface with a project-based structure for managing database resources
- **Autoscaling:** Compute resources automatically adjust based on workload demand. Learn more: Autoscaling
- **Scale-to-zero:** Computes automatically suspend after inactivity, waking up in seconds when needed. Learn more: Scale to zero
- **Projects and branches:** Create instant, isolated copies of your database using copy-on-write technology. Learn more: Branches
- **Instant restore:** Restore your database to any point in time within your configured restore window (2-30 days). Learn more: Point-in-time restore
- **Read replicas:** Create read-only compute endpoints that share the same storage layer, enabling instant creation without data duplication. Learn more: Read replicas
- **High availability:** Pair a primary compute with secondary compute instances across availability zones for automatic failover. Learn more: High availability
- **Data API:** PostgREST-compatible RESTful interface for direct HTTP access to your database. Learn more: Data API

Feature comparison

The following table compares capabilities in Lakebase Autoscaling and Lakebase Provisioned.

> **NOTE**
>
> Lakebase Autoscaling is the new version of Lakebase. New feature development is focused on Lakebase Autoscaling.
>
> New Lakebase instances will be created as Autoscaling projects. Rollout starts March 12, 2026. For details, see Autoscaling by default.

Feature	Autoscaling	Provisioned
Core capabilities		
Provisioned (fixed-size) compute	✓	✓
Autoscaling	✓	
Scale to zero	✓	
Branching	✓	
Instant restore	✓	
Point-in-time restore	✓	✓
Read replicas / readable secondaries	✓ (Read replicas, readable secondaries)	✓ (Readable secondaries)
High availability	✓	✓
Inbound Private Link	✓	✓

Feature	Autoscaling	Provisioned
Inbound Private Link for performance-intensive services	✓	
Compliance security profile	✓ (set to HIPAA, C5, TISAX, or None)	
Customer-managed keys (CMK)	✓	
Data integrations		
Unity Catalog registration	✓	✓
Synced tables (serve lakehouse data with Lakebase)	✓	✓
Lakehouse Sync (sync Lakebase tables to Delta/Iceberg tables)	✓ Beta	
Query federation	✓	✓
Application integrations		
Databricks Apps	✓	✓
Feature Store	✓	✓
Notebooks	✓	✓
Stateful AI agents	✓	✓
Access control		
UI for Postgres role management	✓	✓
Workspace ACLs	✓	✓

Feature	Autoscaling	Provisioned
Developer tools		
Infrastructure as code (Asset Bundles, Terraform)	✓ (Beta)	✓
Programmatic access (REST API, CLI, SDKs)	✓ (Beta)	✓
PostgREST API support	✓ (PostgREST-compatible Data API)	✓ (Private Preview)
Cost management		
Tags and serverless usage policies	✓	✓

Choosing the right version

Both versions of Lakebase are suitable for production workloads. Choose based on your feature requirements:

Choose Lakebase Autoscaling if you want:

- Compute that scales automatically based on workload demand
- Scale-to-zero for cost optimization
- Branch-based development workflows
- Instant point-in-time restore

Choose Lakebase Provisioned if you want:

- Provisioned compute that you manually scale

For detailed feature support differences, see the Feature comparison table above and Lakebase Autoscaling limitations.

this page

as this page helpful?

Yes | No

Send feedback

Ask Genie

*Last updated on **May 12, 2026***

Manage projects

> **INFO**
>
> Lakebase Autoscaling is the latest version of Lakebase, with autoscaling compute, scale-to-zero, branching, and instant restore. For supported regions, see Region availability. If you are a Lakebase Provisioned user, see Lakebase Provisioned.

A project is the top-level container for your Lakebase resources, including branches, computes, databases, and roles. This page explains how to create projects, understand their structure, configure settings, and manage their lifecycle.

If you're new to Lakebase, start with Get started to create your first project.

Understanding projects

Project structure

Understanding the Lakebase project structure helps you organize and manage your resources effectively. A project is the top-level container for your databases, branches, computes, and related resources. Each project includes settings for compute defaults, restore windows, and updates that apply to all branches within the project.

At the top level, a project contains one or more branches. Within a project, you can create branches for different environments such as development, testing, staging, and production. Each branch contains its own computes, roles, and databases.

```
Project
  └── Branches (main, development, staging, etc.)
        ├── Computes (R/W compute)
```

```
        ├── Roles (Postgres roles)
        └── Databases (Postgres databases)
```

Branches

Data resides in branches. Each Lakebase project is created with a root branch called `production`, which cannot be deleted. While you can create additional branches and designate a different branch as your default branch, the root branch cannot be deleted.

You can create child branches from any branch in your project. When you create a child branch, it inherits all databases, roles, and data from its parent branch at the time of creation. Subsequent changes in the parent branch don't automatically propagate to the child branch, enabling isolated development, testing, or experimentatation.

Each branch can contain multiple databases and roles. Learn more: Manage branches

Computes

A compute is a virtualized computing resource that includes vCPU and memory for running Postgres. When you create a project, a primary R/W (read-write) compute is created for the project's default branch. Each branch has a single primary R/W compute. To connect to a database that resides on a branch, you must connect through the R/W compute associated with the branch.

In addition to the primary R/W compute, you can add one or more read replica (read-only) computes to any branch. Read replicas enable you to offload read-only workloads from your primary compute for use cases such as horizontal read scaling, analytics and reporting queries, and read-only access for users or applications. Learn more: Manage computes, Read replicas

Roles

Roles are Postgres roles. A role is required to create and access a database. A role belongs to a branch. When you create a project, a Postgres role is automatically created for your Databricks identity (for example, `user@databricks.com`), which is the owner of the default `databricks_postgres` database. Any role created in the Lakebase

UI is created with `databricks_superuser` privileges. There is a limit of 500 roles per branch. Learn more: Manage roles

Databases

A database is a container for SQL objects such as schemas, tables, views, functions, and indexes. In Lakebase, a database belongs to a branch. The default branch of your project is created with a database named `databricks_postgres`. There is a limit of 500 databases per branch. Learn more: Manage databases

Schemas

All databases in Lakebase are created with a `public` schema, which is the default behavior for any standard Postgres instance. SQL objects are created in the `public` schema by default.

Project limits

Lakebase Postgres enforces the following limits for projects:

Resource	Limit
Maximum number of concurrently active computes	20
Maximum number of read replicas per branch	6
Maximum number of branches per project	500
Maximum number of Postgres roles per branch	500
Maximum number of Postgres databases per branch	500
Database storage quota (per branch)	16 TB
Maximum number of projects per workspace	1000
Maximum number of protected branches	1

Resource	Limit
Maximum number of root branches	3
Maximum number of unarchived branches	10
Maximum number of manual snapshots	10
Maximum history retention period	30 days
Minimum scale to zero time	60 seconds

Concurrently active compute limit

The concurrently active compute limit caps how many computes can run at the same time to prevent resource exhaustion. This limit protects against accidental resource surges, such as starting many compute endpoints at once. The default limit is 20 concurrently active computes per project.

Important: The default branch is exempt from this limit, ensuring it remains available at all times.

When you exceed the limit, additional computes beyond the limit remain suspended and you see an error when attempting to connect to them. To resolve this:

1. Suspend other active computes and try again.
2. If you encounter this error often, contact Databricks Support to request a limit increase.

> **NOTE**
>
> Computes with scale to zero enabled automatically suspend after a period of inactivity, helping you stay within the concurrently active compute limit.

Database storage quota

Each branch has a 16 TB database storage quota.

When a database reaches its quota, write performance drops, but you can still drop or delete data to reclaim space. Contact Databricks Support if you need a larger quota.

Only your actual data (tables and indexes, as reported by Postgres) counts against the quota. History retained for point-in-time restore doesn't.

Region availability

Supported regions:

- `us-east-1` (US East - N. Virginia)
- `us-east-2` (US East - Ohio)
- `us-west-2` (US West - Oregon)
- `ca-central-1` (Canada - Central)
- `sa-east-1` (South America - São Paulo)
- `eu-central-1` (Europe - Frankfurt)
- `eu-west-1` (Europe - Ireland)
- `eu-west-2` (Europe - London)
- `ap-south-1` (Asia Pacific - Mumbai)
- `ap-southeast-1` (Asia Pacific - Singapore)
- `ap-southeast-2` (Asia Pacific - Sydney)

Your Lakebase project is created in your Databricks workspace region.

Postgres version support

Lakebase Postgres Autoscaling supports Postgres 16 and Postgres 17.

Create and manage projects

Create a project

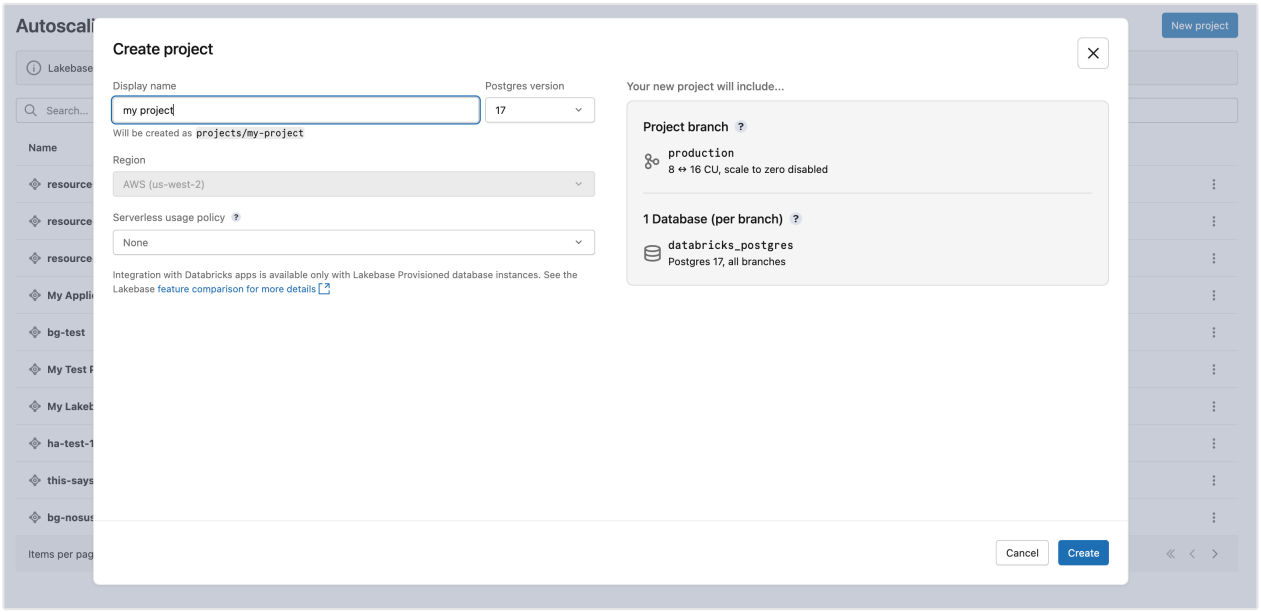
You can create multiple projects in Lakebase Postgres to keep applications or customers fully isolated, ensuring clean separation of data and resources.

To create a project:

UI | Python SDK | Java SDK | CLI | curl

1. Click the apps switcher in the top right corner to open the Lakebase App.
2. Click **New project**.
3. Configure your project settings:
   - **Display name**: Enter a name for your project. You can use any characters, including spaces and special characters. Common naming patterns include naming after the application (for example, `My Analytics App`) or the customer or tenant the project serves (for example, `Acme Corp DB`). A resource name is derived from your display name automatically and is used to identify the project in API and SDK calls. The dialog shows the resulting resource name (for example, `projects/my-analytics-app`) so you can verify it before creating the project.
   - **Postgres version**: Select the Postgres version you want to use.
   - **Serverless usage policy** (optional): Select a serverless usage policy to attribute serverless compute costs to a specific policy. See Serverless usage policies.

The **Create project** dialog shows the project configuration options.



The **Region** for your Lakebase project is set to your Databricks workspace region and cannot be modified.

A new project includes the following resources by default:

- A single `production` branch (the default branch)
- A single primary read-write compute associated with the branch with the following default settings:

Branch	Compute Units (CU)	HA	Autoscaling	Scale-to-zero
`production`	8 - 16 CU	Disabled	Enabled	Enabled (24h)

  When you create a project, the `production` branch is created with a compute that has scale-to-zero enabled by default with a 24-hour inactivity timeout. You can adjust the timeout or disable scale-to-zero for this compute if needed.

- A Postgres database (named `databricks_postgres`)
- A Postgres role for your Databricks identity (for example, `user@databricks.com`)

To change compute settings for an existing project, see Configure project settings. To modify default compute settings for new projects, see **Compute defaults** in Configure project settings.

Get project details

Retrieve details for a specific project.

UI | Python SDK | Java SDK | CLI | curl

1. Click the apps switcher in the top right corner to open the Lakebase App.
2. Select your project from the projects list to view its details.

List projects

List all projects in your workspace.

UI | Python SDK | Java SDK | CLI | curl

1. Click the apps switcher in the top right corner to open the Lakebase App.
2. The projects list displays all projects you have access to.

Configure project settings

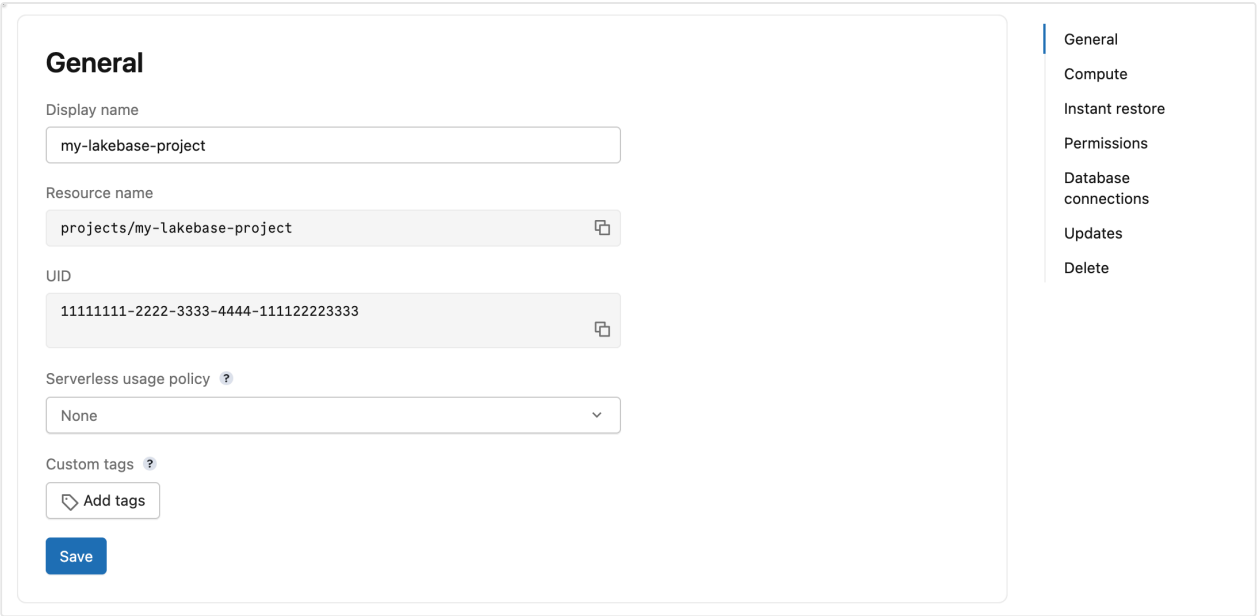
After creating a project, you can modify various settings from the project dashboard by navigating to **Settings**:

General settings

The general settings page displays the following fields:

- **Display name**: The editable display name for your project.
- **Resource name**: Read-only. The full resource path for your project (format: `projects/{project_id}`). Use this value in API and SDK calls to identify the project.
- **UID**: Read-only. The system-generated unique identifier for your project.
- **Serverless usage policy**: Associate a serverless usage policy with your project to attribute serverless compute costs to a specific policy. See Serverless usage policies.
- **Custom tags**: Add key-value tags to your project. Tags are logged in your account's billable usage records (`system.billing.usage`) and can be used to track costs by team, project, or cost center. See Custom tags. When you update custom tags using the API or CLI, the new list replaces all existing tags.

UI | Python SDK | CLI | curl



Compute defaults

These defaults are used as the initial settings for any primary or read replica computes you create. Modifying these defaults does not alter the settings of any existing computes.

Default values:

- **Compute size:** 2 ↔ 4 CU (autoscaling range; ~4–8 GB RAM)
- **Scale to zero:** Enabled by default—**Suspend compute after a period of inactivity** is checked, with **Suspend after 24 hours** selected

Click **Modify defaults** to open the dialog and change these values.

> **NOTE**
>
> To modify settings for an existing compute, see Manage computes.

Lakebase Postgres supports compute sizes from 0.5 CU to 112 CU. Autoscaling is available for computes up to 32 CU (0.5, then integer increments: 1, 2, 3... 16, then 24, 28, 32). Larger fixed-size computes are available from 36 CU to 112 CU (36, 40, 44, 48, 52, 56, 60, 64, 72, 80, 88, 96, 104, 112). Each Compute Unit (CU) provides 2 GB of RAM.

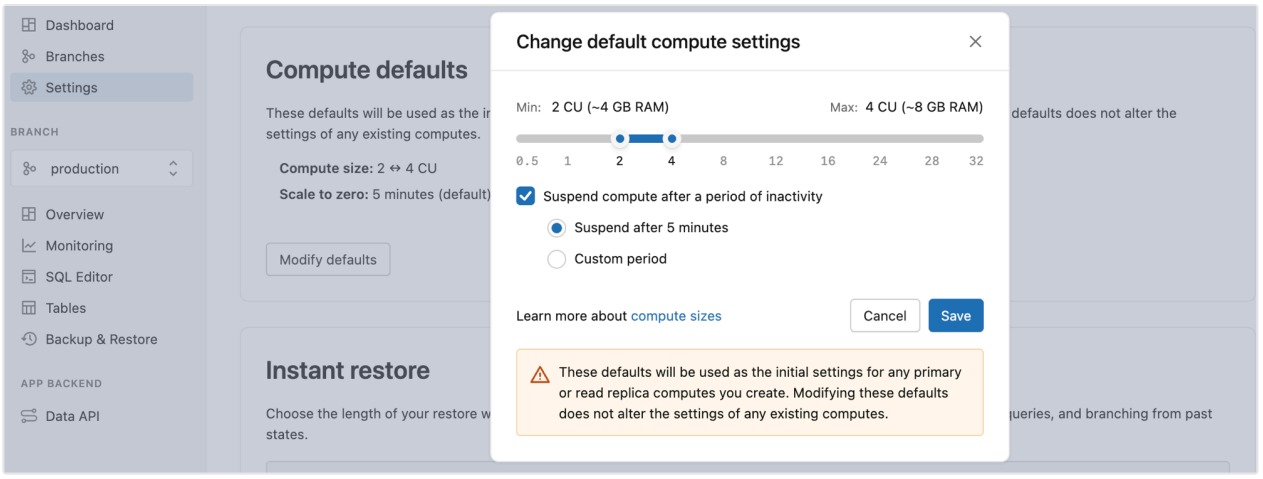
> **NOTE**
>
> Ask Genie

> **Lakebase Provisioned vs Autoscaling**: In Lakebase Provisioned, each Compute Unit allocated approximately 16 GB of RAM. In Lakebase Autoscaling, each CU allocates 2 GB of RAM. This change provides more granular scaling options and cost control.

Representative sizes:

Compute Units	RAM
0.5 CU	1 GB
1 CU	2 GB
4 CU	8 GB
8 CU	16 GB
16 CU	32 GB
32 CU	64 GB
64 CU	128 GB
112 CU	224 GB

- To enable autoscaling, set a compute size range using the slider. Autoscaling dynamically adjusts compute resources based on workload demand. Learn more: Autoscaling
- Adjust the scale-to-zero setting to increase or decrease the amount of inactive compute time before a compute suspends. You can also disable **scale-to-zero** for an always-active compute. Learn more: Scale to zero

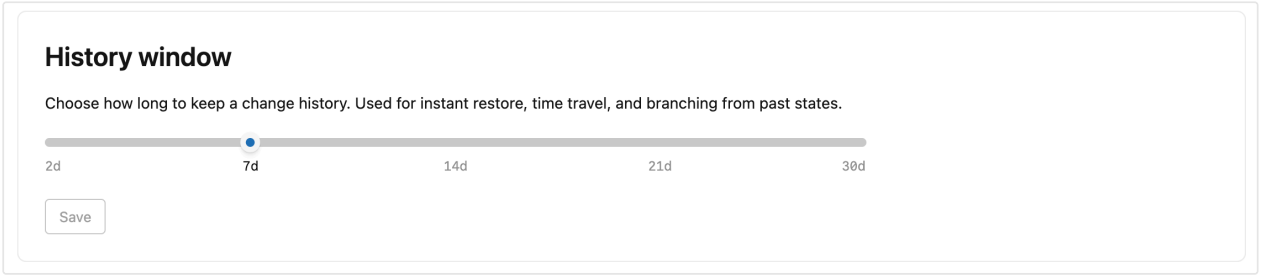


Instant restore

Configure the restore window length for your project. By default, Lakebase retains a history of changes for root branches in your project, enabling point-in-time restore for recovering lost data, querying data at a point in time for investigating data issues, and branching from past states for development workflows.

You can set the restore window from 2 days up to 30 days. Note that:

- Extending the restore window increases your storage
- The restore window setting affects all branches in your project



Project permissions

Control who can access and manage your Lakebase project by granting permissions to Databricks identities, groups, and service principals. Project permissions determine what actions users can perform within the project, such as creating branches, managing computes, and viewing connection details.

Permission types:

- **CAN CREATE**: View and create project resources

- **CAN USE**: View and use project resources (list, view, connect, and perform certain branch operations) without creating or deleting projects or branches
- **CAN MANAGE**: Full control over project configuration and resources

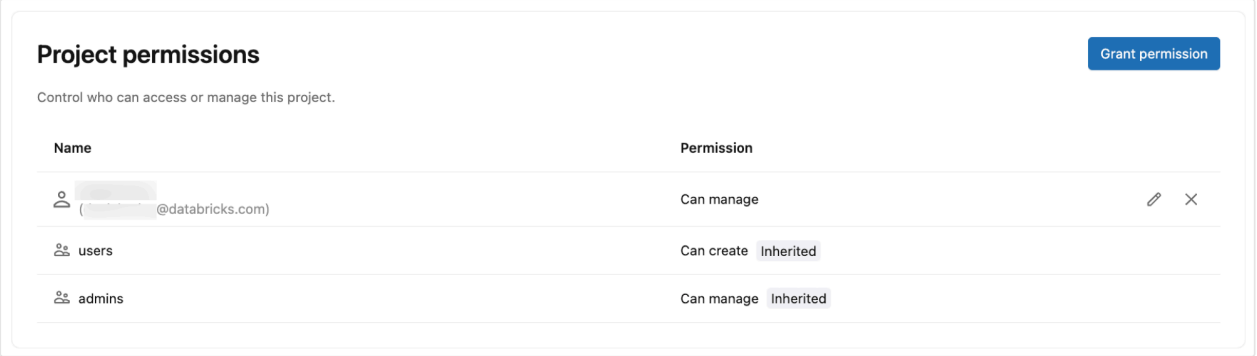
Default permissions:

When you create a project, the following permissions are automatically assigned:

- **Project owner** (the user who created the project): CAN MANAGE (full control)
- **Workspace users**: CAN CREATE (can view and create projects)
- **Workspace admins**: CAN MANAGE (full control)

To grant access to other users, see Manage project permissions.

> **NOTE**
>
> **Project permissions and database access are separate**
>
> Project permissions control Lakebase platform actions, while database access is controlled by Postgres roles and their associated permissions. See Create Postgres roles and Manage database permissions.

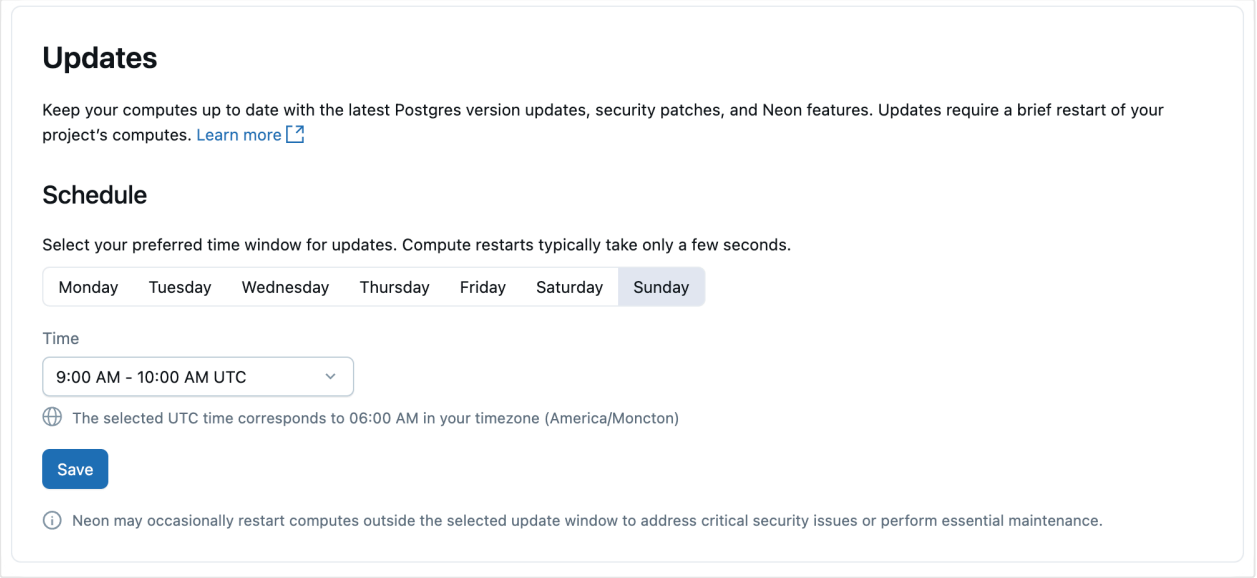


Updates

To keep your Lakebase computes and Postgres instances up to date, Lakebase automatically applies scheduled updates that include Postgres minor version upgrades, security patches, and platform features. Updates are applied to the computes within your project and require a brief compute restart that takes a few seconds.

Updates are applied automatically, but you can set a preferred day and time for updates. Restarts occur within your selected time window.

For detailed information about updates, see Manage updates.



Delete a project

Deleting a project is a permanent action that also deletes any computes, branches, databases, roles, and data that belong to the project.

> **IMPORTANT**
>
> This action cannot be undone. Be cautious when deleting a project, as doing so deletes all associated branches and data.

Before deleting

Databricks recommends deleting all associated Unity Catalog catalogs and synced tables before deleting the project. Otherwise, attempting to view catalogs or run SQL queries that reference them results in errors.

If you are not the owner of the tables or catalogs, you must reassign ownership to yourself before deletion.

> **NOTE**
>
> Ask Genie

> Only users with **CAN MANAGE** permission on the Lakebase project can delete it. See Project ACLs and Manage project permissions for details.

Delete a project

To delete a project:

UI | Python SDK | Java SDK | CLI | curl

1. Navigate to your project's **Settings** in the Lakebase App.
2. In the **Delete project** section, click **Delete** and enter the project name to confirm deletion.

*Last updated on **Apr 30, 2026***

Lakebase Provisioned

> **INFO**
>
> Lakebase Provisioned is the original Lakebase offering that uses provisioned compute you scale manually. For supported regions, see Region availability. For the latest version of Lakebase, with autoscaling compute, scale-to-zero, branching, and instant restore, see Lakebase Autoscaling.
>
> New Lakebase instances will be created as Autoscaling projects. Rollout starts March 12, 2026. For details, see Autoscaling by default.

Lakebase is a fully managed Postgres online transaction processing (OLTP) database engine integrated into the Databricks Data Intelligence Platform. Lakebase lets you create and manage OLTP databases stored in Databricks-managed storage, integrating with your Lakehouse for real-time transactional workloads.

Learn about Lakebase Provisioned

Understand Lakebase architecture, capabilities, and limitations.

Topic	Description
What is Lakebase?	Overview of Lakebase architecture, use cases, and Databricks integration.
Database instance architecture	Learn about database instance components, capabilities, and limitations.
Database role types and permissions	Understand the different Postgres roles and their privileges in Lakebase.

Topic	Description
PostgreSQL compatibility	Learn about PostgreSQL compatibility, limitations, and optimization tips.

Get started with Lakebase Provisioned

For database owners and admins

If you're setting up a new Lakebase database for your team:

Task	Description
Create a database instance	Set up your first Lakebase Provisioned database.
Add users and set permissions	Give other users access to your database and control what they can do.

For database users

If you need to access an existing Lakebase database:

Task	Description
Connect to your database	Get the credentials you need to access your Lakebase database.
Query your data	Use various tools to query your PostgreSQL data including SQL editor, notebooks, and psql client.

Data integration and synchronization

Connect Lakebase with your existing Databricks data and workflows.

Topic	Description
Register with Unity Catalog	Optionally register your PostgreSQL database as a catalog in Unity Catalog for federated queries.
Serve lakehouse data with synced tables	Create synced tables to serve Unity Catalog data through your Lakebase database instance for operational applications.

Advanced features

Explore advanced capabilities for production workloads and enterprise use cases.

Topic	Description
Restore data and time travel	Use child instances to perform time travel and restore data to specific points in time for data recovery, compliance auditing, and development environments.
Monitoring and observability	Monitor your database performance and health using built-in metrics and logging.

Advanced configuration

Explore advanced capabilities for production workloads and enterprise use cases.

Topic	Description
High availability	Configure high availability for your database instances to ensure business continuity.

Topic	Description
Restore window	Set the restore window for your database instance for point-in-time recovery.

*Last updated on **Mar 27, 2026***

What is Lakebase Provisioned?

> **INFO**
>
> Lakebase Provisioned is the original Lakebase offering that uses provisioned compute you scale manually. For supported regions, see Region availability. For the latest version of Lakebase, with autoscaling compute, scale-to-zero, branching, and instant restore, see Lakebase Autoscaling.
>
> New Lakebase instances will be created as Autoscaling projects. Rollout starts March 12, 2026. For details, see Autoscaling by default.

This page introduces Lakebase Provisioned, a fully managed Postgres OLTP database engine, integrated into the Databricks Data Intelligence Platform. A database instance is a type of Databricks compute that provides the storage and compute for running a Postgres server that manages multiple databases.

Overview

An online transaction processing (OLTP) database is a specialized type of database system designed to efficiently handle high volumes of real-time transactional data. Lakebase allows you to create an OLTP database on Databricks, and integrate OLTP workloads with your Lakehouse. This OLTP database enables you to create and manage databases stored in Databricks-managed storage.

Using an OLTP database in conjunction with the Databricks platform significantly reduces application complexity. Lakebase is well integrated with Databricks Feature Store, SQL warehouses, and Databricks Apps. Using sync tables provides a simple and performant way to sync data between OLTP and online analytical processing (OLAP) workloads.

> **NOTE**
>
> This page describes Lakebase Provisioned. For the latest version of Lakebase, which supports autoscaling compute, scale-to-zero, branching, and instant restore, see Lakebase Autoscaling.

Region availability

Supported regions:

- `us-east-1`
- `us-east-2`
- `us-west-2`
- `eu-central-1`
- `eu-west-1`
- `ap-south-1`
- `ap-southeast-1`
- `ap-southeast-2`

*Last updated on **May 11, 2026***

Lakebase Autoscaling

> **INFO**
>
> Lakebase Autoscaling is the latest version of Lakebase, with autoscaling compute, scale-to-zero, branching, and instant restore. For supported regions, see Region availability. If you are a Lakebase Provisioned user, see Lakebase Provisioned.

Lakebase Autoscaling is 100% standard Postgres with automatic scaling, instant branching, and deep integration with the Databricks platform. Create a project, get a connection string, and start building.

Get started with Lakebase

Create a project, connect, run your first query, and explore integrations with Unity Catalog and the Databricks lakehouse.

Serve lakehouse data in your app

Sync Unity Catalog tables into Lakebase for low-latency app serving. Eliminate reverse ETL pipelines with fully managed syncs.

Replicate operational data to the lakehouse

Stream Postgres changes to Delta tables with continuous CDC for analytics, history, and downstream pipelines. (Beta)

Learn about Lakebase Autoscaling

What is Lakebase Autoscaling?

Overview of Lakebase architecture, branching, and how projects are organized.

Tutorials

Hands-on tutorials for common workflows including branching, app development, and user access.

Core concepts

Autoscaling, scale-to-zero, branching, and how separated compute and storage works.

Key features

Explore features that optimize performance, reduce costs, and enable flexible development workflows.

Autoscaling

Automatically adjust compute resources based on workload demand.

Scale to zero

Automatically suspend inactive computes to minimize costs.

Branches

Create isolated branches for development and testing.

Read replicas

Create read-only replicas to scale read operations.

Instant restore

Create a new branch from any point in time within your restore window.

Connect and query

Use various tools and interfaces to connect to and query your database. Ask Genie

Connect to your database

Learn different ways to connect to your Lakebase database.

Query with SQL Editor

Use the built-in SQL Editor to query and manage your database.

Tables editor

Use the visual interface to view, edit, and manage data and schemas.

Postgres clients

Connect using standard Postgres clients and tools.

Querying data at a point in time

Query data using point-in-time branches.

Databricks integrations

Connect Lakebase with your existing Databricks data and workflows.

Register in Unity Catalog

Register your Lakebase database in Unity Catalog for unified governance.

Serve data with synced tables

Serve lakehouse data through your Lakebase database for low-latency applications.

Lakehouse Sync

Continuously sync Lakebase tables to Unity Catalog Delta tables for analytics and history. (Beta)

Manage and operate

Create, configure, and manage your projects and resources.

Manage projects

Create, configure, and manage projects.

Manage project permissions

Grant and manage permissions to control project access and management.

Manage branches

Create and manage branches for development and testing.

Manage computes

Configure compute resources and scaling for your branches.

Manage read replicas

Create and manage read-only replicas to scale read operations.

Manage roles

Create and manage Postgres roles and permissions.

Manage databases

Create and manage databases within your branches.

API

Use the Lakebase Autoscaling API to manage projects, branches, and computes programmatically.

Databricks CLI

Use the Databricks CLI to manage projects, branches, and computes programmatically.

Declarative Automation Bundles

Use Declarative Automation Bundles to manage projects, branches, and computes as code.

Terraform

Use Terraform to manage projects, branches, and computes as code.

Updates

Manage Postgres version updates for your projects.

Monitor

Track performance, usage, and health metrics.

Backup and restore

Configure automated backups and point-in-time recovery.

this page

as this page helpful?

Yes | No

Send feedback

Ask Genie

*Last updated on **May 11, 2026***

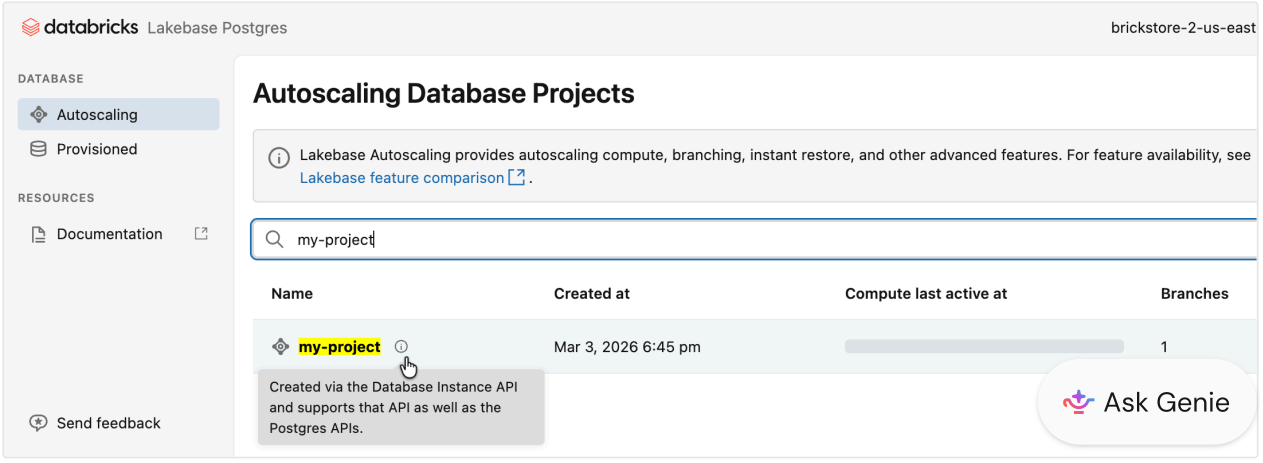
Autoscaling by default

> **NOTE**
>
> Starting **March 12, 2026**, new Lakebase instances will be created as Lakebase Autoscaling projects (instead of Provisioned instances) so you can use capabilities such as autoscaling compute, scale-to-zero, branching, and instant restore.
>
> The rollout begins on March 12 and is expected to complete within a few days. Until your workspace is updated, you can still create Provisioned instances. Once your workspace is in the rollout, any new Lakebase instance you create will be an Autoscaling project.

When you create a new Lakebase instance, it is now created as a Lakebase Autoscaling project and appears on the **Autoscaling** page in the Lakebase app, not on the **Provisioned** page. Existing Provisioned instances are not affected and continue running exactly as before.

On the **Autoscaling** page, the new projects are marked with an information icon.

Hover over the icon and you'll see a tooltip explaining that the project was created using the Database instance API, and that it supports both Database instance and Postgres APIs.



What this means for you

- **Lakebase instance creation.** Any new Lakebase instance is created as a Lakebase Autoscaling project and appears on the **Autoscaling** page in the Lakebase app. You can manage these projects with both the Database instance API and the Postgres API.
- **Create button in the UI.** The **Create** button (including in the Provisioned tab) now opens the Autoscaling project creation flow.
- **Autoscaling features.** New projects get autoscaling compute, scale-to-zero (enabled by default with a 24-hour inactivity timeout), branching, and instant restore. You can access these features from the Lakebase Autoscaling UI and Postgres API when you're ready.
- **Existing Lakebase Provisioned instances are unchanged.** They stay on the **Provisioned** page in the UI with the same behavior as before. Existing Provisioned instances keep running, connection strings keep working, and APIs remain operational.
- **Your existing automation continues to work.** Your existing automation keeps working without modification, but new instances are created as Autoscaling projects.

> **NOTE**
>
> At a future date, you will be able to upgrade Provisioned instances to Lakebase Autoscaling projects. Databricks will communicate the upgrade timeline as it approaches.

APIs

To ensure your existing automation developed on Lakebase Provisioned continues to operate without interruption, both the Database Instance API (Lakebase Provisioned) and the Postgres API (Lakebase Autoscaling) work with newly created Lakebase instances. The following table shows which API to use, depending on how your project or instance was created.

How the project or instance was created	API to use
New projects created using the Database instance API or the Lakehouse/Provisioned UI	Both the Database instance API and the Postgres API work. You can manage the same project with either API.
New and existing projects created only in the Autoscaling UI or with the Postgres API	Postgres API only.
Existing Lakebase Provisioned database instances	Database instance API only.

If you use the Feature Store or sync tables and provision them programmatically (for example, online feature stores), use the Database instance API for now. Programmatic provisioning of sync tables is not yet supported on the Postgres API. This gap will close in a future release.

> **NOTE**
> Lakebase API support will eventually converge on the Postgres API. Databricks will communicate the timeline for moving away from the Database instance API as it approaches.

Differences for new Lakebase instances

If you're used to Lakebase Provisioned, new Lakebase instances created as Lakebase Autoscaling projects have some special characteristics. Here's what to expect.

Compute size

Lakebase instances created as Autoscaling projects use Autoscaling **compute units (CUs)** instead of Lakebase Provisioned capacity units. When a project is created using the Database instance API, your chosen Lakebase Provisioned capacity unit is mapped to a Lakebase Autoscaling min and max autoscaling CU range as follows:

> ⓘ NOTE
>
> **RAM per unit:** In Lakebase Provisioned, one capacity unit has 16 GB RAM. In Lakebase Autoscaling, one CU has 2 GB RAM.

Provisioned capacity	Autoscaling min CU	Autoscaling max CU
1 (16 GB)	4 (8 GB)	8 (16 GB)
2 (32 GB)	8 (16 GB)	16 (32 GB)
4 (64 GB)	16 (32 GB)	32 (64 GB)
8 (128 GB)	64 (128 GB)	64 (128 GB)

Lakebase Autoscaling supports **autoscaling only up to 32 CU**. Larger sizes (including 64 CU) are fixed-size. So for capacity 1-4, you get a min-max autoscaling range. For capacity 8, you get a fixed 64 CU compute (min and max both 64).

For example, if your automation creates Lakebase instances with **capacity 1**, new Lakebase Autoscaling projects are created with **min CU 4** and **max CU 8** settings. You can change min/max later using the Postgres API or in the Lakebase Autoscaling UI. If you set max CU above 64 using the Postgres API, the Database instance API will report capacity as 8.

> ⓘ PRICING
>
> All new instances, including those created using the Database instance API, run on the Autoscaling platform and use Lakebase Autoscaling pricing. See the Lakebase pricing page for details.

> Existing Provisioned instances remain on their current pricing until they are upgraded to Autoscaling. Databricks will communicate the upgrade timeline as it approaches.

Scale-to-zero

The read/write endpoint for new projects has **scale-to-zero enabled** by default with a **24-hour inactivity timeout**, so your compute automatically suspends after 24 hours of inactivity to reduce costs. You can adjust the timeout, or disable scale-to-zero to keep compute always running, in the Lakebase Autoscaling UI or using the Postgres API. See Scale to zero for details.

Instant restore (restore window)

On **Lakebase Provisioned** (database instances), the **restore window** can be set up to **35 days**. On **Lakebase Autoscaling** (new projects), the **instant restore** setting has a maximum of **30 days**. If you rely on a restore window longer than 30 days on Provisioned, note that new projects support a maximum of 30 days for point-in-time recovery, time travel, and branching from past states.

PostgreSQL version

New projects created through the Database instance API or Lakehouse/Provisioned UI use **PostgreSQL 16**, aligned with Lakebase Provisioned. That means automation using the Database instance API creates projects with the same PostgreSQL version (16) as before. The default for projects created using the Autoscaling UI or Postgres API is **PostgreSQL 17**.

Connection details and hostnames

Connection details for new projects use a different **hostname format** than Lakebase Provisioned. Provisioned instances use a **global** hostname (no region in the hostname). Newly created projects use a **regional** hostname (region included). Both read-write and read-only endpoints use the regional hostname format for new projects.

Provisioned (global hostname):

```
host=instance-a1b2c3d4-e5f6-7890-abcd-
ef1234567890.database.cloud.databricks.com
```

Autoscaling (regional hostname):

```
host=ep-example-endpoint-a1b2c3d4.database.<region>.cloud.databricks.com
```

The regional hostname includes the region code for your cloud and region (for example, `us-east-1` on AWS or `eastus` on Azure).

> **IMPORTANT**
>
> If you use IP allowlists, you must allow the regional ingress IPs for your region for new projects. See Create IP access lists for workspaces.

Private Link

Lakebase Autoscaling uses two Private Link endpoints: **front-end Private Link** for API access (workspace-level connectivity. Most customers already have this, no change) and **inbound Private Link for performance-intensive services** (also called regional front-end Private Link) for connecting Postgres clients to the database. Inbound Private Link for performance-intensive services is in Public Preview. If you use Private Link and create a new Lakebase instance, you only need to add inbound Private Link for performance-intensive services if you do not already have it. Front-end Private Link is unchanged. Existing Provisioned instances keep their current Private Link configuration.

Your situation	What you need
You use Private Link and create a new instance, and you do not yet have inbound Private Link for performance-intensive services	Add inbound Private Link for performance-intensive services.
You already use Lakebase Autoscaling or other services that use inbound Private Link for performance-intensive services	No change.

Your situation	What you need
You only use existing Provisioned instances and do not create new ones	No change.

Permissions (ACLs)

Permissions on new Lakebase instances are stored on the Lakebase project ACL resource. You can manage them through the standard Permissions API with `request_object_type=database-projects`, the Lakebase Autoscaling UI, or the Postgres API. Existing automation that uses the Database instance API continues to work because those calls are proxied to the same `database-projects` resource, so reads and writes stay consistent across both API surfaces.

Earlier permissions model

Project names and instance names

Project and instance names must be **DNS compliant**. Name uniqueness is **case insensitive**, which means you cannot have both `ABC` and `abc` in the same workspace. When using the Postgres API, use the lowercase project name. Creation fails if the name collides with an existing project ID.

Branch and project rename

For projects created using the Database instance API or the Lakehouse/Provisioned UI, **branch rename and project rename are not available**. The name you use in the Database instance API is the name of the project and branch in the Autoscaling UI.

Child database instances

For **child** database instances (a child instance corresponds to a branch on the Autoscaling project):

- **Branch limit.** With the Postgres API or the Autoscaling UI, you can create any number of branches. With the Database instance API, you can create only one parent instance (one root branch) and one child instance (one child branch), which matches the previous Database instance API behavior.
- **Tags and budget.** Custom tags and budget policies are **inherited from the parent** branch. This is a change from the previous Database instance API behavior, where child instances had their own separate values.
- **Deleting a child instance.** If you create additional branches using the Lakebase Autoscaling UI or Postgres API on a project that corresponds to a child instance, you cannot delete that child instance through the Lakehouse/Provisioned UI or API until all of those branches are deleted. Delete the branches first (using the Autoscaling UI or Postgres API), then delete the child instance.

Protected branches

If you protect a branch on a project created using the Database instance API (whether the branch corresponds to a child instance or any other branch you created), you cannot delete that child instance or the root (parent) instance through the Lakehouse/Provisioned UI or API until the branch is unprotected. Unprotect the branch in the Lakebase Autoscaling UI first, then delete the instance. See Protected branches.

Private preview features

Newly created projects are Lakebase Autoscaling projects and do not support the Lakebase Provisioned **Data API** or **Forward ETL** private preview features. Those were separate offerings on Provisioned.

- **REST API access (PostgREST-style):** Lakebase Autoscaling has its own Data API for PostgREST-compatible REST access (CRUD, query, RPC).
- **Syncing data to the lakehouse:** Lakebase Autoscaling has Lakehouse Sync, which is different from the private preview Forward ETL on Lakebase Provisioned.

Frequently asked questions (FAQ) about Autoscaling by default

Does this apply to both AWS and Azure?

Yes. The change applies to both AWS and Azure. The rollout begins on the same date on both clouds.

Where do newly created Lakebase instances appear?

On the **Autoscaling** page in the Lakebase app, not on the **Provisioned** page. See the intro above.

What happens when I click Create in the Lakebase UI?

The **Create** button (including in the Provisioned tab) opens the Autoscaling project creation flow. You can no longer create new Provisioned instances from the UI. If you have no Provisioned instances, the Provisioned tab is hidden. See What this means for you.

Are all features that were available on Provisioned also supported on Autoscaling?

Most capabilities available on Lakebase Provisioned are supported on Lakebase Autoscaling, with some differences. For a full side-by-side comparison, see the Feature comparison table on the Lakebase index page.

What does NOT change?

The following are unchanged:

- **Existing Provisioned instances:** they keep running with their current connection strings, pricing, and Private Link configuration.
- **Existing Autoscaling instances:** no change.
- **The Database instance API:** it remains fully supported. You do not need to change existing Terraform, SDKs, or other automation that you developed for

Lakebase Provisioned.

- **Existing DABs automation using `database_instances`:** Bundles using the `database_instances` resource continue to work. However, new instances created by `database_instances` are created as Lakebase Autoscaling projects. For new Lakebase work, we recommend using `postgres_projects` instead. See bundle resources.
- **Connection strings for existing Lakebase Provisioned instances:** they remain unchanged. No updates required.
- **Private Link configuration** for existing Provisioned or Autoscaling instances. No change.

How does this change impact pricing?

All new instances (including those created using the Database instance API) use Lakebase Autoscaling pricing. See Compute size.

Can I still use the Database instance API for new projects?

Yes, for new projects created using the Database instance API or Lakehouse/Provisioned UI. Both APIs work for these newly created projects. See APIs.

Can I use the Postgres API for my existing Provisioned instances?

No. Use the Database instance API only for existing Lakebase Provisioned instances. See APIs.

Can I enable scale-to-zero or set min/max CU with the Database instance API?

New projects created with the Database instance API have scale-to-zero enabled by default with a 24-hour inactivity timeout. To adjust the timeout, disable scale-to-zero, or set the autoscaling compute range (min/max CU), use the Postgres API or the

Lakebase Autoscaling UI. The Database instance API does not expose these controls. See Compute size and Scale-to-zero.

Why do the capacity numbers look different (for example, capacity 1 vs 4-8 CU)?

Autoscaling uses compute units (CUs) with a different RAM-per-unit than Provisioned capacity. See Compute size.

What PostgreSQL version do new projects use?

Projects created from the Lakehouse/Provisioned UI or the Database instance API use PostgreSQL 16, same as Lakebase Provisioned. Automation using the Database instance API therefore gets the same PostgreSQL version (16) as before. See PostgreSQL version.

Do I need to change my connection details or hostnames?

New projects use regional hostnames. If you use IP allowlists or Private Link, you need to update them for newly created projects. See Connection details and hostnames.

Do I need to change my Private Link setup for new instances?

Most customers do not need to change anything. You only need to add **inbound Private Link for performance-intensive services** if you use Private Link, create new Lakebase instances, and do not already have inbound Private Link for performance-intensive services configured. If you already use Lakebase Autoscaling or other services that use inbound Private Link for performance-intensive services, or you only use existing Provisioned instances, no change is needed. See Private Link for the full table and setup links.

Do my existing Provisioned instances need any Private Link changes?

No. Existing Provisioned instances keep their current Private Link configuration. No reconfiguration is required. See Private Link.

What's the difference between the restore window (Provisioned) and instant restore (Autoscaling)?

Lakebase Provisioned supports a restore window of up to 35 days. New Lakebase Autoscaling projects use instant restore with a maximum of 30 days. See Instant restore (restore window).

How do instance and project names work for new Lakebase instances?

Names must be DNS compliant. Uniqueness is case insensitive (you cannot have both `ABC` and `abc`). When using the Postgres API, use the lowercase project name. Creation fails if the name collides with an existing project ID. See Project names and instance names.

Do my existing Provisioned instances change?

No. They stay on the **Provisioned** page with the same behavior. Only new Lakebase instances are created as Autoscaling projects.

How do permissions work on projects created from the Lakehouse/Provisioned UI or Database instance API?

Permissions are managed through Lakebase project ACLs. You can manage them in the Lakebase Autoscaling UI or through the Postgres API. Existing automation that uses the Database instance API continues to work. See Permissions (ACLs).

Can I use PostgREST or get REST API access to my new project?

The Data API (PostgREST-style) was a private preview feature on Lakebase Provisioned. Newly created projects use the Lakebase Autoscaling Data API for PostgREST-compatible REST access (CRUD, query, RPC). See Private preview features.

Learn more

- What is Lakebase Autoscaling?
- Connection strings and About authentication
- Create a project

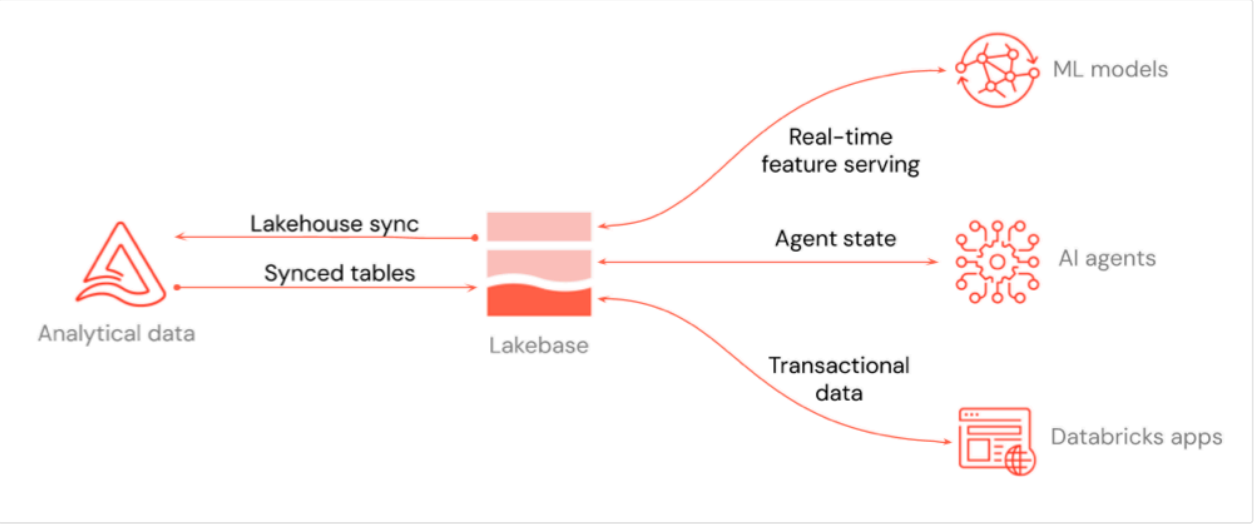
*Last updated on **May 4, 2026***

What is Lakebase Autoscaling?

> **INFO**
>
> Lakebase Autoscaling is the latest version of Lakebase, with autoscaling compute, scale-to-zero, branching, and instant restore. For supported regions, see Region availability. If you are a Lakebase Provisioned user, see Lakebase Provisioned.

Lakebase Postgres Autoscaling is a fully managed Postgres database built for any application that requires online transaction processing (OLTP) and low-latency data serving. It is integrated into the Databricks platform, enabling you to build real-time transactional applications alongside your analytics workloads.

Lakebase Postgres Autoscaling combines the reliability and familiarity of Postgres with modern database capabilities including autoscaling, scale-to-zero, branching, and instant restore. These features enable flexible development workflows, cost-efficient operations, and rapid iteration.



The diagram shows how Lakebase integrates with the rest of the platform: real-time feature serving for ML models and Feature Store, agent state for AI agents, and transactional data for Databricks Apps or any application you connect to it.

You can move data in either direction between your lakehouse and Lakebase. **Synced tables** move data from the lakehouse into Lakebase so your applications can query it at low latency.

Lakehouse Sync moves data from your OLTP application in Lakebase back into the lakehouse for analytics and downstream processing.

Example use cases and workload types

The following are just a few examples of the many ways you can use an OLTP Postgres database like Lakebase across industries: personalized recommendations and offer targeting in e-commerce and retail, clinical trial data and recommendation systems in healthcare, automated trading and streaming analytics in financial services, and machine telemetry and maintenance workflows in manufacturing.

Common workload types for OLTP databases may include the following:

- **Data serving:** Serve insights from golden tables to applications at low latency and high QPS.
- **Store application state:** Manage workflow and agent state in a transactional data store.
- **Feature serving:** Serve featurized data at low latency to ML models.

Databricks integration

The diagram above highlights three key integration use cases:

- **Real-time feature serving:** Use Lakebase projects as an online store for ML models and Feature Store, so you can serve featurized data at low latency. See Online Feature Store (Lakebase) and Feature Serving.
- **Agent state for AI agents:** Store and manage state for AI agents in a transactional database, so conversations and workflow context persist across requests.
- **Transactional data for applications:** Persist data for Databricks Apps or any application you connect to Lakebase. For Databricks Apps, add a Lakebase project

as an app resource. See Add a Lakebase resource to a Databricks app.

Lakebase Provisioned

Lakebase Provisioned is the original Lakebase offering that uses provisioned compute you scale manually. Existing Provisioned instances continue to be supported. New Lakebase development is focused on Autoscaling. If you have Provisioned instances or are evaluating both options, see What is Lakebase Provisioned? and Autoscaling by default.

What is a project?

Lakebase Autoscaling resources are organized into a **project** structure. A project is the top-level container for your database resources. When you create a Lakebase Autoscaling database, you create a project. The project holds your branches (database environments), computes, roles, and databases. Think of a project as the unit of organization for one application or workload. You can have multiple projects in a workspace, each with its own branches and data.

How projects are organized

Understanding the hierarchy of objects within a project helps you organize and manage your resources:

```
Databricks Workspace
  └── Project(s)
      └── Branch(es)
          ├── Compute (primary R/W)
          ├── Read replica(s) (optional)
          ├── Role(s)
          └── Database(s)
                └── Schema(s)
```

Each level in the hierarchy serves a specific purpose:

Object	Description
Project	The top-level container for your database resources. A project contains branches, databases, roles, and compute resources. See Manage projects.
Branch	An isolated database environment that shares storage with its parent branch. Each project can contain multiple branches. See Manage branches.
Compute	The Postgres server that powers a branch. Each branch has its own compute that provides the processing power and memory for database operations. See Manage computes.
Database	A standard Postgres database within a branch. Each branch can contain multiple databases with their own tables, schemas, and data. See Manage databases.

Understanding branches

One of Lakebase Postgres's most powerful features is branching. Like Git branches for your code, branches let you create isolated database environments for development and testing—without affecting production.

Why this matters: Traditional database workflows require separate dev and staging servers, manual data refreshes, and careful coordination. With branches, you can:

- Instantly create a development environment with production data
- Test schema changes safely before applying them to production
- Recover from mistakes by creating branches from any point in time
- Pay only for the data you change, not full duplicate databases

Topic	Description
Branches	Learn how branches work, common workflows, and best practices for your team.
Manage branches	Create, reset, and delete branches for development and testing.
Protected branches	Protect production branches from accidental changes and deletions.

Core concepts

Lakebase is built on several key innovations that differentiate it from traditional database systems:

- **Separated compute and storage:** Scale compute resources independently from storage for cost efficiency and flexibility.
- **Autoscaling:** Compute automatically adjusts based on workload demand, with support for scale-to-zero during idle periods.
- **Copy-on-write storage:** Enables instant branching where you only pay for data changes, not full duplicates.
- **Instant point-in-time operations:** Create branches or restore to any moment within your configured restore window (2-30 days)

These concepts work together to enable flexible development workflows, cost-efficient operations, and rapid recovery from mistakes.

For a detailed explanation of each core concept, see Core concepts.

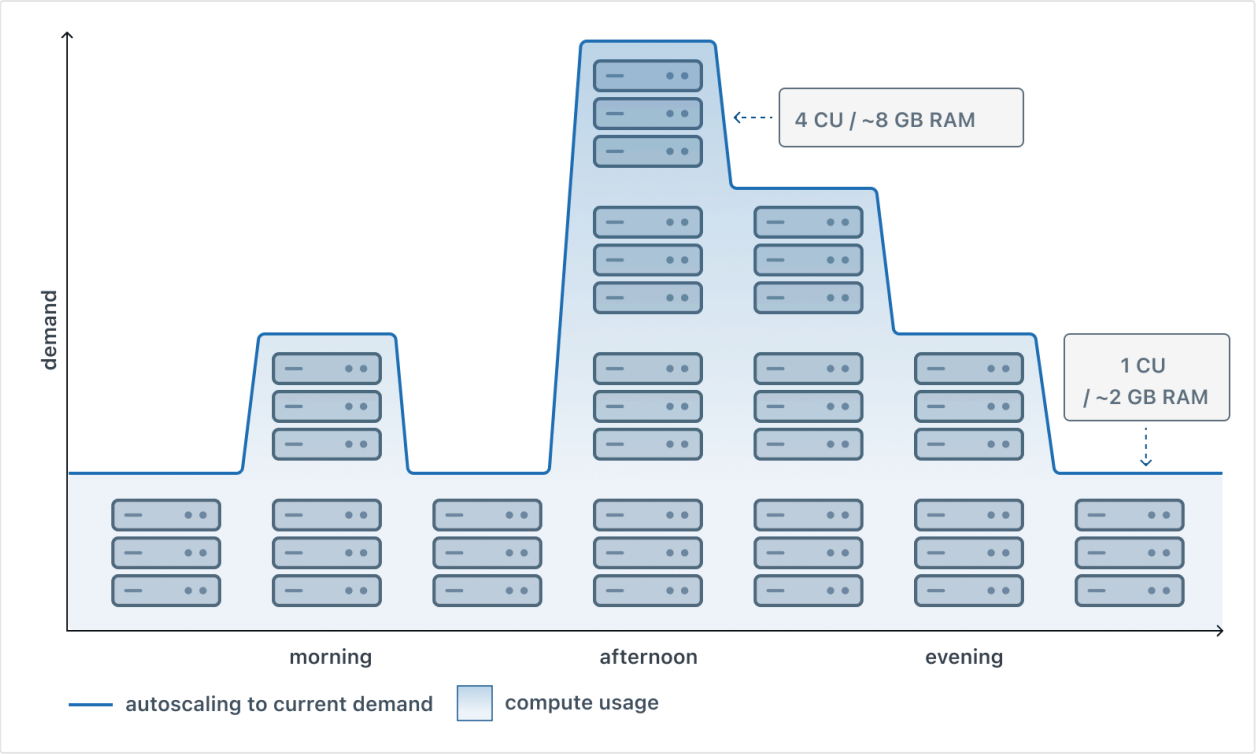
*Last updated on **Apr 10, 2026***

Autoscaling

> **INFO**
>
> Lakebase Autoscaling is the latest version of Lakebase, with autoscaling compute, scale-to-zero, branching, and instant restore. For supported regions, see Region availability. If you are a Lakebase Provisioned user, see Lakebase Provisioned.

Autoscaling dynamically adjusts the amount of compute resources allocated to your Lakebase computes in response to current workload demands. As your application experiences varying levels of activity throughout the day, autoscaling automatically increases compute capacity during peak usage and reduces it during quieter periods, eliminating the need for manual intervention.

This visualization shows how autoscaling works throughout a typical day, with compute resources scaling up or down based on demand to ensure your database has the resources it needs while conserving resources during off-peak times.



Autoscaling operates within a user-defined range. For example, you might set a compute to scale between 2 and 8 Compute Units (CU), with each CU providing 2 GB of RAM. Your compute automatically adjusts within these limits based on workload, never dropping below the minimum or exceeding the maximum regardless of demand. Autoscaling is available for computes up to 32 CU.

> **NOTE**
>
> **Lakebase Provisioned vs Autoscaling**: In Lakebase Provisioned, each Compute Unit allocated approximately 16 GB of RAM. In Lakebase Autoscaling, each CU allocates 2 GB of RAM. This change provides more granular scaling options and cost control.

How autoscaling works

Automatic resource adjustment

When you enable autoscaling and set your minimum and maximum compute sizes, Lakebase continuously monitors your workload and adjusts resources automatically. The system tracks three key metrics to make scaling decisions:

- **CPU load:** Monitors processor utilization to ensure your database has adequate processing power.
- **Memory usage:** Tracks RAM consumption to prevent memory constraints.
- **Working set size:** Estimates your frequently accessed data to optimize cache performance.

Based on these metrics, Lakebase scales your compute up when demand increases and scales down when activity decreases, all while staying within your configured range.

Scaling boundaries

You define the scaling range by setting minimum and maximum compute sizes. This range provides:

- **Performance guarantees:** The minimum ensures baseline performance even during low activity.
- **Cost control:** The maximum prevents unbounded resource consumption and costs.
- **Automatic optimization:** Within these boundaries, Lakebase handles all scaling decisions.

The difference between your maximum and minimum cannot exceed 16 CU (that is, `max − min ≤ 16 CU`).

No downtime or manual intervention

Autoscaling adjustments within your configured range happen without requiring compute restarts or connection interruptions. However, changing the minimum or maximum CU configuration may cause a brief interruption to active connections. Once configured, the system operates autonomously, allowing you to focus on your applications rather than infrastructure management.

Autoscaling benefits

Cost efficiency: You only pay for the compute resources you actually use. During off-peak hours, your compute scales down, reducing costs. During peak periods, it scales up to maintain performance.

Performance optimization: Your database automatically receives additional resources when workload increases, preventing performance degradation during traffic spikes or intensive operations.

Predictable costs: By setting a maximum compute size, you control the upper bound of your compute costs, preventing unexpected expenses from runaway resource consumption.

Simplified operations: Autoscaling eliminates the need to manually monitor workload patterns and adjust compute sizes, reducing operational overhead and the risk of human error.

Configuring autoscaling

Autoscaling configuration requires setting minimum and maximum compute size boundaries. Autoscaling is available for computes up to 32 CU. For workloads requiring more than 32 CU, larger fixed-size computes from 36 CU to 112 CU are available.

For detailed instructions on enabling and configuring autoscaling, see Manage computes.

Common autoscaling scenarios

AI agent and application workloads

AI agents and interactive applications built on Databricks often experience variable request patterns. Autoscaling ensures your database handles traffic spikes during active user sessions while reducing costs during quiet periods.

For details on connecting Lakebase with Databricks AI and application services, see Databricks integrations.

Development and testing environments

Development branches for testing schema changes or validating data pipelines typically see intermittent activity. Autoscaling minimizes resources during idle periods while ensuring adequate performance during active development.

Customer-facing dashboards and applications

Applications delivering analytics or operational insights to end users often have time-of-day usage patterns. Autoscaling automatically adjusts resources to match user activity throughout the day.

Autoscaling and scale to zero

Autoscaling works in combination with scale to zero. While autoscaling adjusts resources based on workload demand, scale to zero suspends a compute entirely after a period of inactivity, reducing compute costs to zero during idle periods.

When you configure both features:

1. **Active period:** Autoscaling adjusts compute size based on workload within your defined range.
2. **Inactive period:** After the scale-to-zero timeout, the compute suspends entirely.
3. **Resumed activity:** The compute restarts at the minimum autoscaling size when new queries arrive.

This combination maximizes cost efficiency, particularly for development, testing, or staging environments that experience long idle periods.

Autoscaling and high availability

Autoscaling is supported for high availability endpoints. CU size adjustments apply uniformly to all computes in a high availability configuration — the autoscaling range you configure applies to the primary and all secondaries together.

Two constraints apply when autoscaling is combined with high availability:

- **Secondaries cannot scale below the primary's current CU size.** This ensures secondaries are always ready to take over as primary without a performance gap after promotion.
- **Scale to zero is not available for computes in a high availability configuration.** To reduce costs during inactivity, consider using scale to zero on non-HA branches instead.

Next steps

- Manage computes to learn how to enable and configure autoscaling
- Metrics dashboard to view CPU, RAM, and working set size metrics
- Scale to zero to understand how computes can suspend during inactivity
- High availability to understand how autoscaling works in a high availability configuration
- Database branches to learn about creating isolated database environments
- Databricks integrations to connect Lakebase with Databricks AI and application services

*Last updated on **May 6, 2026***

Scale to zero

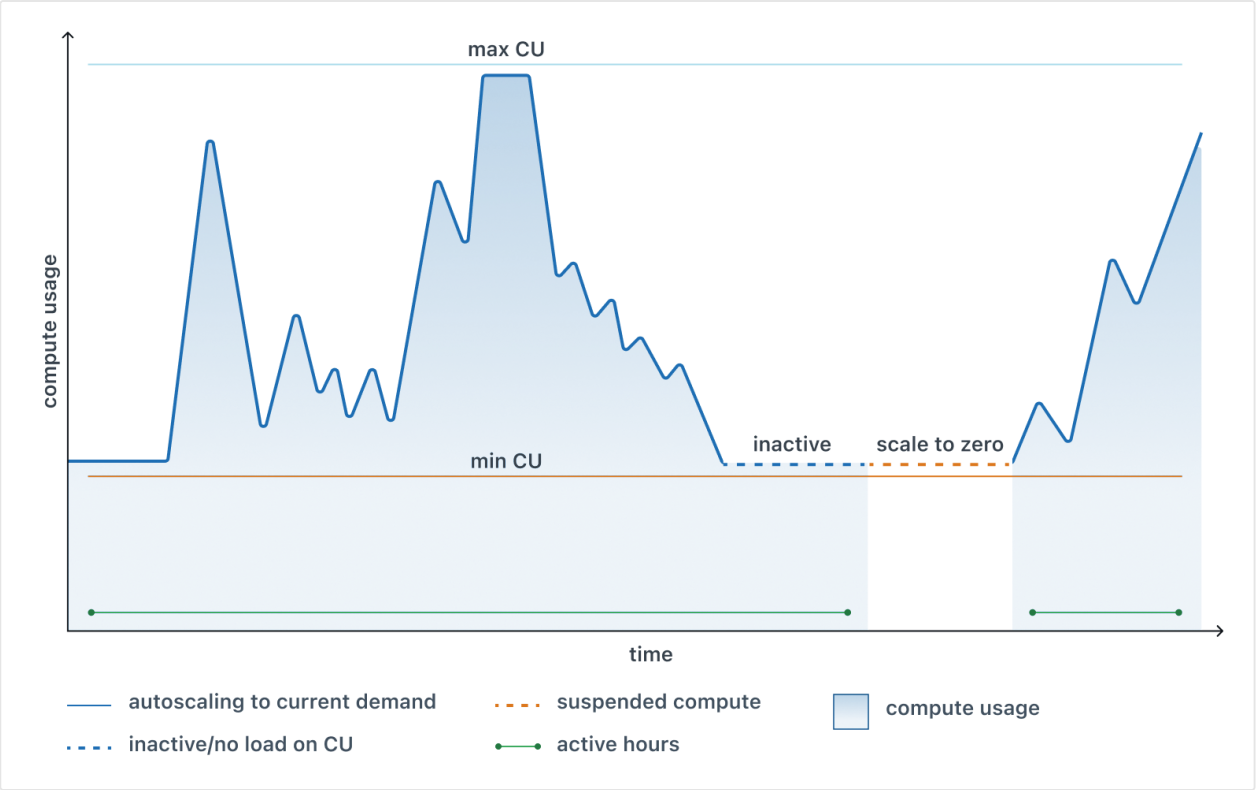
> **INFO**
>
> Lakebase Autoscaling is the latest version of Lakebase, with autoscaling compute, scale-to-zero, branching, and instant restore. For supported regions, see Region availability. If you are a Lakebase Provisioned user, see Lakebase Provisioned.

Scale to zero automatically suspends your Lakebase compute after a period of inactivity, minimizing costs for databases that aren't continuously active. This feature is particularly valuable for development, testing, and staging environments, as well as production databases with predictable idle periods.

When scale to zero is enabled:

- Your compute automatically suspends after a period of inactivity (default is 24 hours, minimum is 60 seconds)
- You pay only for active compute time, not for idle periods
- The compute automatically reactivates within a few hundred milliseconds when you run a new query

This diagram illustrates scale to zero behavior alongside autoscaling, showing an inactive period followed by automatic suspension until the database is accessed again.



Scale to zero works independently from autoscaling. While autoscaling adjusts compute resources during active periods based on workload demand, scale to zero suspends the compute entirely during inactivity, reducing compute costs to zero.

How scale to zero works

Automatic suspension

When your compute remains idle—receiving no queries or connections—for the configured timeout period, Lakebase automatically suspends it. During suspension:

- The compute consumes no resources and incurs no compute costs
- Your data remains safely stored and available
- Connection strings and credentials remain valid
- The compute endpoint remains accessible but inactive

Automatic reactivation

When a new query or connection request arrives at a suspended compute, Lakebase automatically reactivates it. The reactivation process:

- Requires no manual intervention
- Transparently handles the connection request once active
- Restores the compute to its configured minimum size (if autoscaling is enabled)

Applications should implement connection retry logic to handle the brief reactivation period gracefully.

Timeout configuration

You configure the scale-to-zero timeout to control how quickly a compute suspends after becoming idle. The timeout determines the balance between:

- **Shorter timeouts (60 seconds - 5 minutes)**: Faster suspension reduces costs but may cause more frequent reactivations for intermittent workloads
- **Longer timeouts (5 minutes - 1 hour)**: Fewer reactivations improve user experience for sporadic activity but may increase costs during extended idle periods

The minimum timeout is 60 seconds. The maximum is configurable based on your use case.

Scale to zero benefits

- **Cost reduction:** By suspending inactive computes, you pay only for actual usage time. A development database used 8 hours per day costs one-third as much as an always-active compute.
- **Flexible deployment:** Scale to zero enables cost-effective deployment of multiple environments. You can maintain separate development, testing, staging, and preview environments without incurring 24/7 compute costs for each.
- **No manual management:** The system automatically handles suspension and reactivation, eliminating the need to manually start and stop computes based on usage patterns.

- **Preserved configuration:** All compute settings, connection details, and database configurations remain intact during suspension. When the compute reactivates, it resumes with the same configuration.

Configuring scale to zero

Scale to zero can be enabled or disabled for any compute. When enabled, you configure the inactivity timeout that triggers suspension (default is 24 hours, minimum is 60 seconds).

A common configuration is for production branches to have scale to zero disabled for continuous availability, while development branches have it enabled to optimize costs.

For detailed instructions on configuring scale-to-zero settings, see Manage computes.

Common scale to zero scenarios

Development and testing environments

Development branches for testing schema changes, validating data pipelines, or experimenting with new features typically see intermittent activity. Scale to zero automatically suspends these computes during evenings, weekends, and between work sessions, significantly reducing costs.

Staging and preview environments

Staging environments used for pre-deployment validation or preview environments created for pull requests often remain idle between testing cycles. Scale to zero ensures these environments consume resources only during active testing periods.

AI agents and applications with idle periods

AI agents, chatbots, or internal tools that serve specific business hours or have predictable downtime patterns can benefit from scale to zero. The compute suspends during off-hours and reactivates automatically when users return.

Multi-tenant application databases

Applications serving multiple customers can use scale to zero for tenant-specific databases. Computes for inactive tenants suspend automatically, reducing aggregate compute costs across all tenants.

Important considerations

Session context reset

When a compute suspends and later reactivates, the session context resets. This includes:

- In-memory statistics and cache contents
- Temporary tables and prepared statements
- Session-specific configuration settings
- Connection pools and active transactions

If your application requires persistent session data, consider disabling scale to zero to maintain continuous compute availability.

Startup latency

The brief reactivation period (typically a few hundred milliseconds) may impact user experience for the first query after suspension. For applications requiring immediate response times, you can:

- Disable scale to zero for always-available computes
- Implement application-level connection warming
- Use longer timeout periods to reduce reactivation frequency

Production branch behavior

When you create a project, the `production` branch is created with scale to zero enabled by default, with a 24-hour inactivity timeout. You can adjust the timeout or

disable scale to zero for the production branch if needed.

Scale to zero and autoscaling

Scale to zero complements autoscaling to optimize both performance and costs:

- **During active periods:** Autoscaling adjusts compute size based on workload demand within your configured range, scaling up during high activity and down during lighter loads.
- **During inactive periods:** After the scale-to-zero timeout, the compute suspends entirely and compute costs drop to zero regardless of the configured autoscaling range.
- **When reactivated:** The compute restarts at the minimum autoscaling size (if autoscaling is enabled), and autoscaling then adjusts resources based on the new workload.

This combination maximizes efficiency: autoscaling optimizes resource usage during activity, while scale to zero eliminates costs during inactivity.

Next steps

- Manage computes to learn how to configure scale-to-zero settings
- Metrics dashboard to view how metrics reflect inactive compute periods
- Autoscaling to understand how computes adjust resources during active periods
- Database branches to learn about creating isolated database environments

*Last updated on **Mar 10, 2026***

Branches

> **INFO**
>
> Lakebase Autoscaling is the latest version of Lakebase, with autoscaling compute, scale-to-zero, branching, and instant restore. For supported regions, see Region availability. If you are a Lakebase Provisioned user, see Lakebase Provisioned.

Branching in Lakebase lets you version, test, and evolve your data environment safely, similar to branching your code in Git. You can instantly create isolated, fully functional branches for development, experimentation, or testing schema changes, without impacting production workloads.

By default, Lakebase creates a single `production` branch when you create a new project. This is your default branch, meant to house your application's production data.

You can create additional branches as needed to fit your workflow. For example, add a `development` branch for building and testing, a `staging` branch for pre-production testing, or create per-developer branches for complete isolation. Each branch operates independently—changes in a child never affect its parent. With branch reset, you can refresh any child branch from its parent to get the latest schema and data, without data seeding or teardown scripts.

How branches work

Parent-child relationships

Every branch (except the root branch) has a parent. This creates a hierarchy:

```
production (root branch)
├── staging (child of production)
```

```
│   └── feature-test (child of staging)
└── development (child of production)
    └── bugfix-branch (child of development)
```

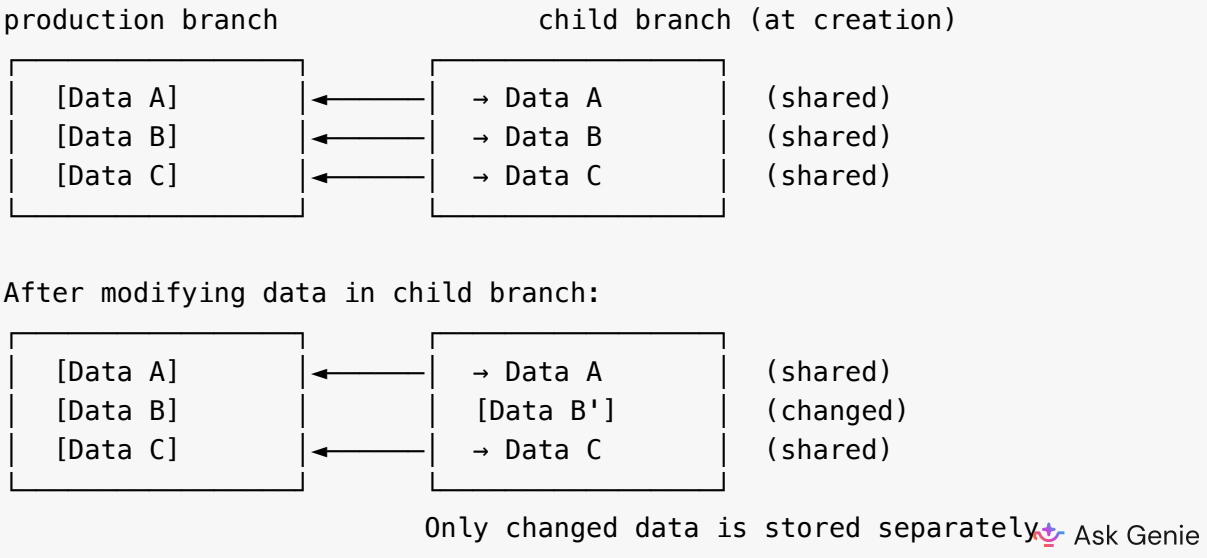
This hierarchy gives you important isolation: changes you make to a child branch don't affect its parent, and changes to a parent don't automatically appear in children. When you need updated data from the parent, you can reset the child branch. You can also create branches from any point in the parent's history, which is useful for point-in-time recovery, testing against historical data, or compliance scenarios.

When you create a branch, you choose whether to initialize it from current data or from a specific point in time. See Create a branch for step-by-step instructions and details on each option.

Copy-on-write storage

Lakebase uses copy-on-write technology to make parent-child branching efficient. When you create a new branch, it inherits both the schema and data from its parent, but shares the underlying storage through pointers to the same data. Only when you modify data does Lakebase write new data. This means:

- Your branches appear instantly; the size of your database has no impact on branch creation time
- You only pay for data that actually changes between branches
- Creating branches has no performance impact on your production workload



Working with branches

Branch reset

Branch reset instantly updates a child branch to match its parent's current state. This is useful when you want to refresh your development or staging branch with the latest data from its parent. The operation completes instantly using copy-on-write technology, and your connection details remain the same.

Branch reset only works one direction (parent → child). To move changes from child to parent, use your standard migration tools to apply schema changes. See Reset a branch for detailed steps and scenarios.

Point-in-time recovery

You can create a branch from a specific point in time within your restore window, which is useful for recovering from data errors like accidental deletions, investigating past issues, or accessing historical data for auditing and compliance purposes. For example, if a critical table was dropped yesterday at 10:23 AM, you can create a branch set to 10:22 AM to extract the missing data. Similarly, you can create branches reflecting your database state at specific dates for financial reconciliations, regulatory audits, or forensic analysis. Unlike branch reset (which updates an existing branch in-place), point-in-time recovery creates a new root branch from historical data while leaving your original branch unchanged and operational. See Point-in-time restore for details.

Special branch types

Default branch

Each Lakebase project is created with a default branch called `production`, but you can designate any branch as the default. The default branch is exempt from the concurrently active compute limit, ensuring it remains available at all times.

Protected branches

Protected branches have safeguards to prevent accidental changes. They cannot be deleted or reset from their parent, and they're exempt from automatic archival due to inactivity. Protected branches also block project deletion while they exist, ensuring you can't accidentally remove critical infrastructure. Use protected branches for critical data like production. See Protected branches for details.

How branching impacts resource consumption

With branches, you only pay for what you actually use.

Storage: You only pay for data that changes. If you create a development branch and modify 1GB of data in a 100GB database, you pay for approximately 1GB of storage, not 200GB. The unchanged 99GB is shared between branches.

Compute: Each branch has its own compute that you can scale independently. You pay only for active compute hours. Computes scale to zero when idle. This means a development branch you use occasionally costs far less than running a dedicated development server 24/7.

Default branch: Your default branch compute never scales to zero, ensuring your production workload remains available.

Branch strategies

Here are some common ways teams organize their branches:

Simple (individuals and small teams)

Use your default branch with a single development branch:

```
production
└── development
```

Your `development` branch is where you build new features safely. You can make schema changes, add test data, and experiment without any risk to your production branch. When you're ready, run your tested schema migrations against `production` (using your migration tool), then reset `development` to start the next feature with fresh data.

With staging

Add a staging branch for pre-production testing:

```
production
├── staging
└── development
```

If you need pre-production testing, maintain a `staging` branch that mirrors your production branch data. Deploy your application there, run integration and performance tests against realistic data, and gain confidence before going live. Periodically reset `staging` from `production` to refresh your test data.

Per-developer branches

Each developer works in complete isolation:

```
production
└── development
    ├── dev-alice
    ├── dev-bob
    └── dev-charlie
```

This pattern prevents developers from interfering with each other's work and allows everyone to test schema changes independently. Each developer can experiment with their own schema changes and data modifications without affecting others, then apply tested migrations to the shared `development` or `production` branch when ready.

Next steps

- Manage branches to learn how to create, reset, and delete branches

- Protected branches to safeguard critical branches

this page

as this page helpful?

Yes | No

Send feedback

Ask Genie

*Last updated on **May 12, 2026***

Point-in-time restore

> **INFO**
>
> Lakebase Autoscaling is the latest version of Lakebase, with autoscaling compute, scale-to-zero, branching, and instant restore. For supported regions, see Region availability. If you are a Lakebase Provisioned user, see Lakebase Provisioned.

You can restore from any point in time within your project's restore window. This restore option lets you quickly recover from an accidental data loss or modification such as an unintended deletion or schema change.

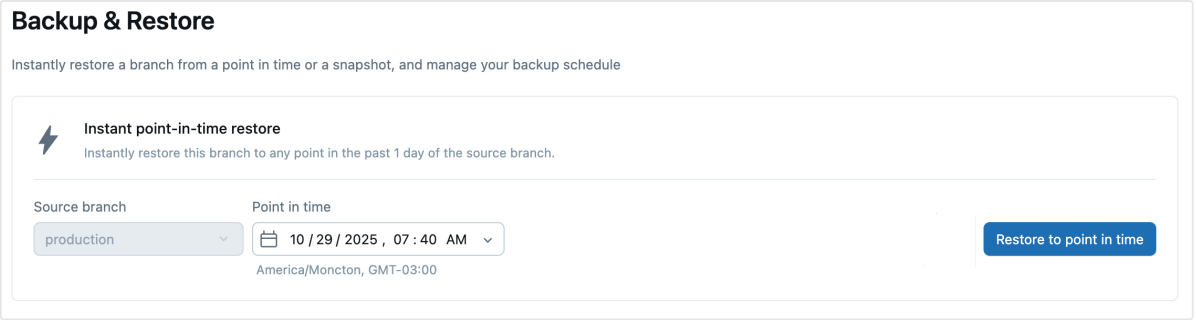
When to use point-in-time restore

Point-in-time restore is best for unexpected events like data loss, unintended deletions, or accidental schema changes. Use this method when you need to quickly restore to a specific moment in time to recover from recent issues.

Perform a restore operation

To perform a restore operation:

1. In the Lakebase App, navigate to your project and select **Backup & Restore** from the sidebar.
2. Under **Instant point-in-time restore**, select your source branch and choose a restore point using the date & time picker. Optionally, click **Preview data** to verify the branch state before proceeding.



3. Click **Restore to point in time**, review the confirmation details, then click **Restore**.

What happens after a restore?

When you perform a point-in-time restore, Lakebase creates a new root branch with your data as it existed at the selected moment. Here's what happens:

- **A new root branch is created** containing your data from the specified point in time.
- **Your original branch remains unchanged** and continues to operate normally.
- **Existing connections continue working** with your original branch without interruption.

The restored branch is created as a root branch, which means you can configure snapshot schedules and other root-branch features on it. Projects have a maximum of 3 root branches, so you may need to delete unused root branches before performing additional restores.

If you want to use the restored data, update your application connection configuration to point to the new branch.

This approach lets you safely recover data without disrupting active operations on your original branch.

Configure your restore window

Lakebase retains a history of changes for root branches in your project. This history enables point-in-time restore for recovering lost data, querying data at a point in time

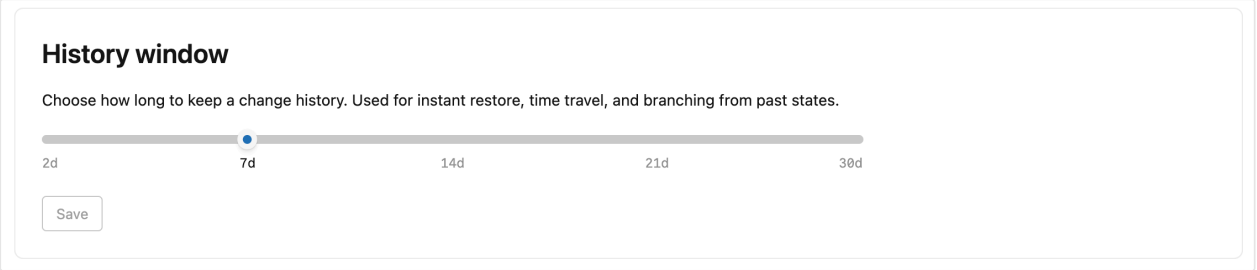
for investigating data issues, and branching from past states for development workflows.

You can configure the restore window from 2 days up to 30 days, with a default of 7 days. The restore window defines how far back you can restore. Note that:

- Extending the restore window increases your storage
- The restore window setting affects all branches in your project

To configure the restore window for a project:

1. Navigate to your project in the Lakebase App and select **Settings** on the **Project dashboard**.
2. Select **History window** and use the slider to set the retention period from 2 days up to 30 days.



Restore details

This section outlines key details about point-in-time restore.

- Creates a new branch, not in-place modification
- All databases on a branch are restored
- Connections remain unchanged

Creates a new branch, not in-place modification

Point-in-time restore creates a new branch containing your data from the specified point in time. This is *not* an in-place modification of your existing branch. The new branch contains a complete copy of all data and schema as they existed at the restore

point. Your original branch remains unchanged and continues operating normally with all its current data.

Restores apply to all Postgres databases

Each branch is an instance of Postgres. A Postgres instance can have more than one database. Keep this in mind when performing point-in-time restores. For example, if you want to recover lost data in a given database and you create a new branch from a point in time before the data loss occurred, the new branch includes *all* Postgres databases from that point in time, not just the one you're troubleshooting.

Connections remain unchanged

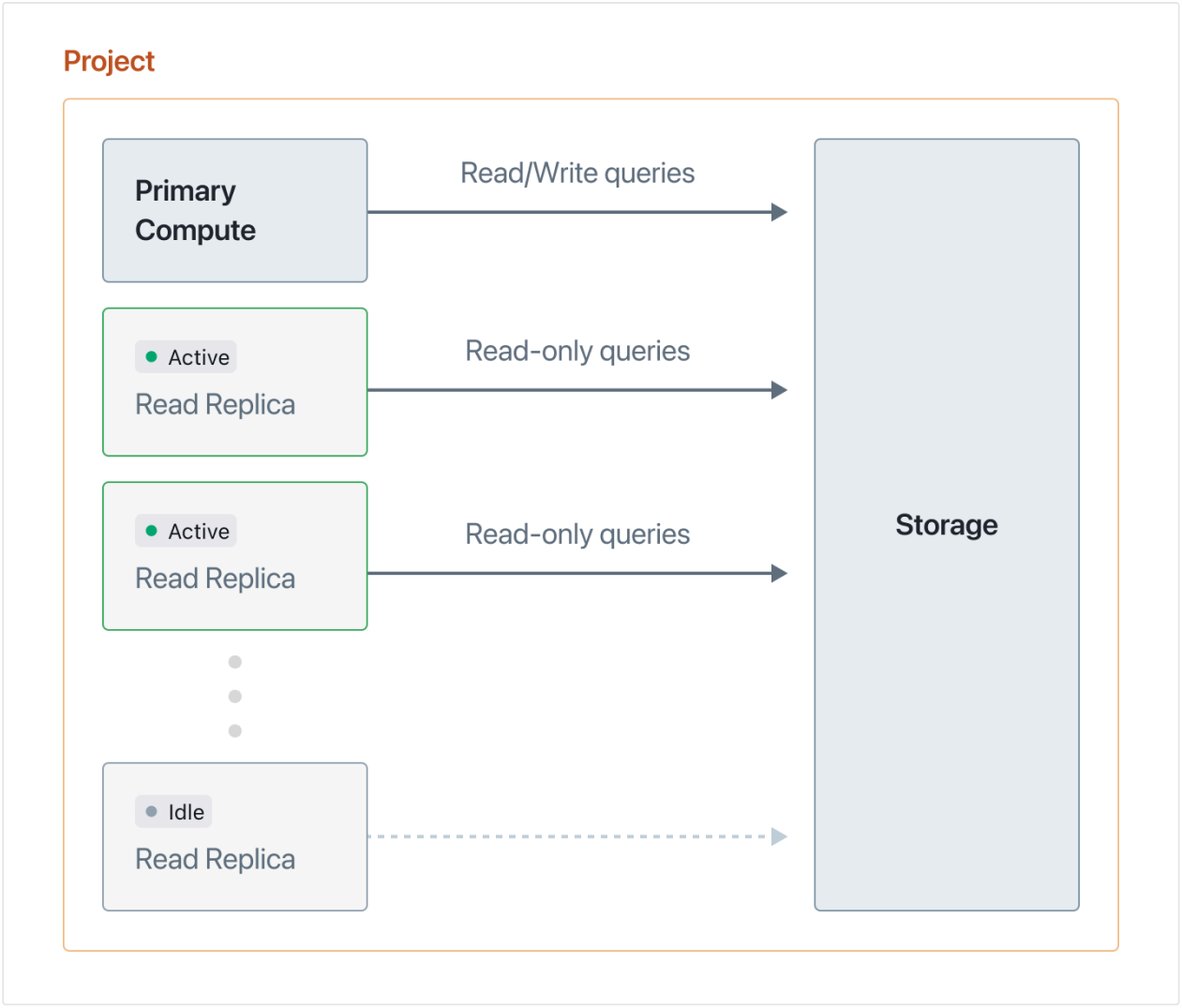
Point-in-time restore does not affect existing connections to your original branch. Since the restore creates a new branch rather than modifying your existing branch, all applications and connections continue to work with your original branch without any interruption. To use the restored data, you must manually update your application connection configuration to point to the new branch.

Last updated on ***Mar 24, 2026***

Read replicas

> **INFO**
>
> Lakebase Autoscaling is the latest version of Lakebase, with autoscaling compute, scale-to-zero, branching, and instant restore. For supported regions, see Region availability. If you are a Lakebase Provisioned user, see Lakebase Provisioned.

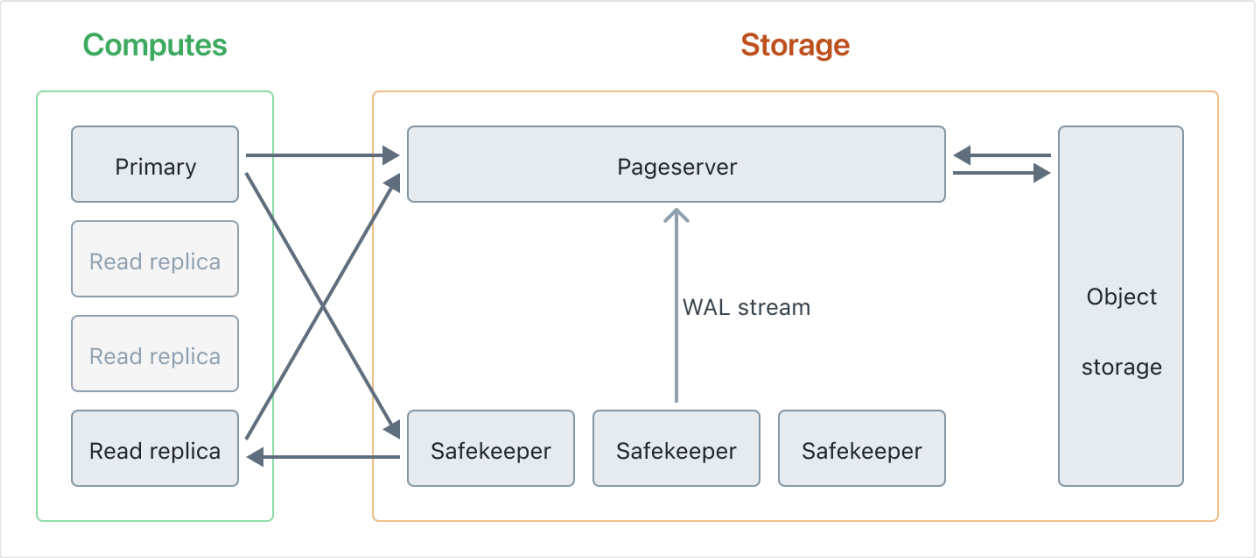
Read replicas are independent read-only computes that perform read operations on the same data as your primary read-write compute. Lakebase read replicas don't replicate or duplicate data. Instead, read requests are served from the same storage layer. While your read-write queries are directed through your primary compute, read queries can be offloaded to one or more read replicas.



You can create read replicas for any branch in your project and configure the size of the compute allocated to each. Read replicas support Lakebase's autoscaling and scale-to-zero features, providing you with the same control over compute resources that you have with your primary compute.

Read replica architecture

Your primary compute and read replicas send read requests to the same storage layer, ensuring that all computes read from a consistent data source. Read replicas are asynchronous, which means they are *eventually consistent*. As updates are made by your primary compute, the storage layer durably stores data changes and keeps read replica computes up to date with the most recent changes to maintain data consistency.



Use cases

Lakebase read replicas have several key applications:

- **Horizontal scaling:** Distribute read requests across replicas to improve performance and increase throughput.
- **Analytics and reporting queries:** Offload resource-intensive analytics and reporting workloads to reduce load on your primary compute.
- **Read-only access:** Grant read-only access to users or applications that don't require write permissions.

Benefits of Lakebase read replicas

- **No additional storage required:** Read replicas read from the same storage as your primary compute with no data duplication or replication.
- **Create them almost instantly:** With no data replication required, read replicas can be created in seconds.
- **Cost-efficient:** No additional storage costs, plus benefit from autoscaling and scale-to-zero features.
- **Instantly available:** Read replicas scale to zero without lag and are immediately up to date when they start.

Create and manage read replicas

You can create read replicas using the Lakebase App. From the **Computes** tab on a branch, click **Add Read Replica** to instantly provision a new read replica compute.

> **NOTE**
>
> You can add up to 6 read replicas per branch. See Project limits.

For detailed instructions on creating and managing read replicas, see Manage read replicas.

Related information

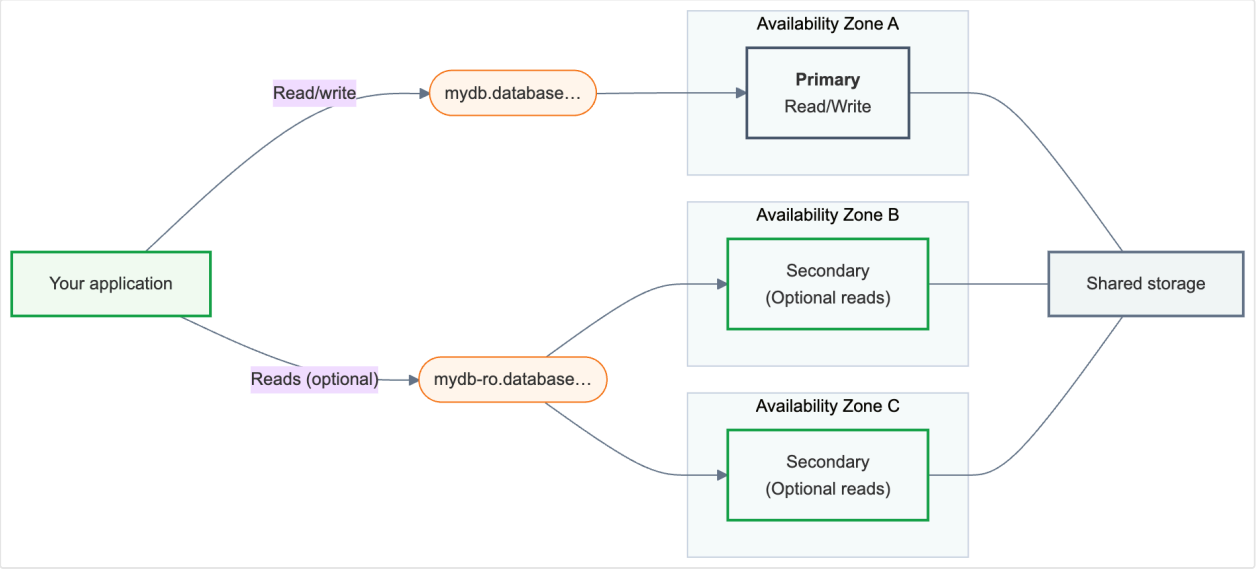
- Manage read replicas
- Metrics dashboard to view replication delay metrics for read replicas
- Autoscaling
- Scale to zero

*Last updated on **Mar 27, 2026***

High availability

> **INFO**
>
> Lakebase Autoscaling is the latest version of Lakebase, with autoscaling compute, scale-to-zero, branching, and instant restore. For supported regions, see Region availability. If you are a Lakebase Provisioned user, see Lakebase Provisioned.

High availability pairs a primary read/write compute with one or more secondary compute instances distributed across availability zones. When the primary becomes unavailable, a secondary compute instance is automatically promoted and your application continues from the last committed transaction. Your connection string remains unchanged.



How high availability works

A **Lakebase endpoint** is the database address your application connects to. A high availability endpoint exposes two connection strings:

- **Primary** (`{endpoint-id}.database.{region}.databricks.com`) — your main read/write connection. Use this in every application that connects to your database. After a failover, it automatically routes to whichever compute is now primary.
- **Secondary** (`{endpoint-id}-ro.database.{region}.databricks.com`) — only available when **Allow access to read-only compute instances** is enabled. Secondary compute instances exist primarily as failover standbys; enabling read access lets you additionally route read queries across them.

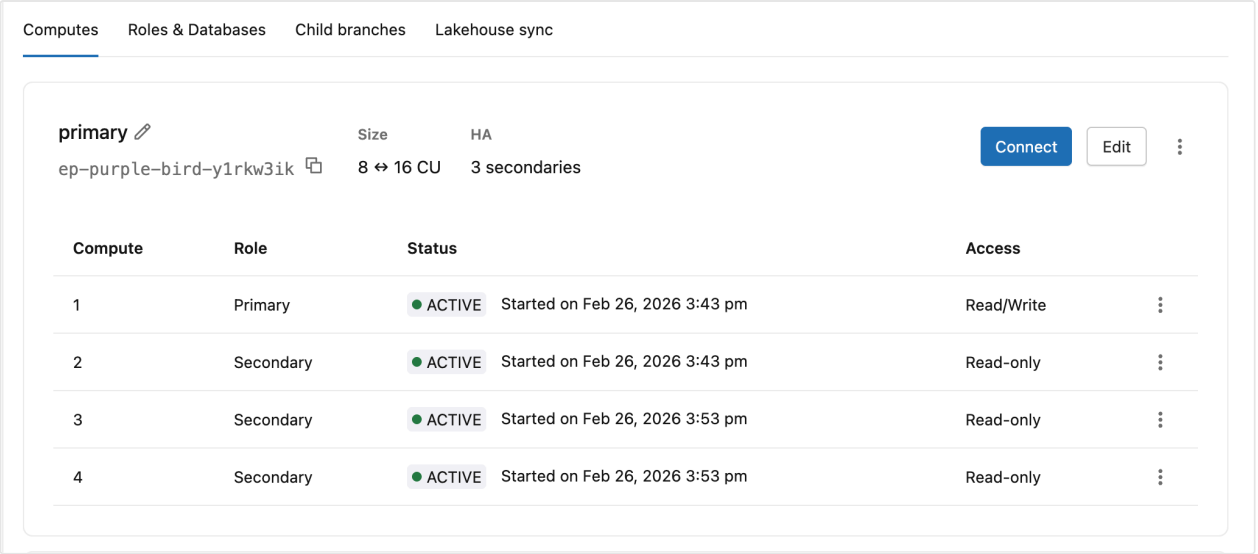
Both connection strings are available from the **Connect** dialog on your endpoint.

Behind these connection strings, a high availability endpoint always has exactly one **Primary** compute instance and one to three **Secondary** compute instances. The primary handles all read/write traffic. Secondary compute instances run in different availability zones and are promoted to become the primary in the event of a failure.

Each secondary compute instance has an **Access** setting that determines whether it also serves read traffic:

Secondary access	What it does
Read-only	Secondary compute instance serves reads via the `-ro` connection string and can be promoted as primary as needed
Disabled	Secondary compute instance is active and ready for failover, but doesn't serve read traffic

You control this with the **Allow access to read-only compute instances** setting on the endpoint, which you can access in the **Edit compute** drawer. When enabled, all secondary compute instances serve reads; when disabled, they are on standby for failover only. In both cases, compute hardware is already allocated and running: promotion requires no new provisioning, so your failover capacity is reserved regardless of demand in the availability zone.



The Computes tab shows each compute instance's **Role** (Primary or Secondary), **Status**, and **Access** level at a glance.

AZ distribution

Lakebase distributes the primary and secondary compute instances across availability zones, reducing the risk that a single AZ failure affects both the primary and all secondary compute instances.

Autoscaling in high availability

All compute instances in a high availability configuration share the same autoscaling range. The maximum spread between your minimum and maximum CU is 16 CU, the same limit as standalone compute instances.

Secondary compute instances are always scaled to at least the same CU size as the primary, ensuring your database capacity stays consistent after a failover.

Scale to zero is not available for compute instances in a high availability configuration. You can manually pause all compute instances, but your endpoint will be unavailable while paused.

Secondary compute instances vs. standalone read replicas

Secondary compute instances and standalone read replicas are different features that can coexist on the same branch:

	Secondary compute instances	**Standalone read replicas**
Purpose	Failover + optional read offload	Read offload only
Added via	High availability configuration	Add Read Replica
Participates in failover	Yes	No
Connection string	`-ro` on the primary endpoint	Own separate endpoint
Sizing	Shared with primary (endpoint-level)	Sized independently

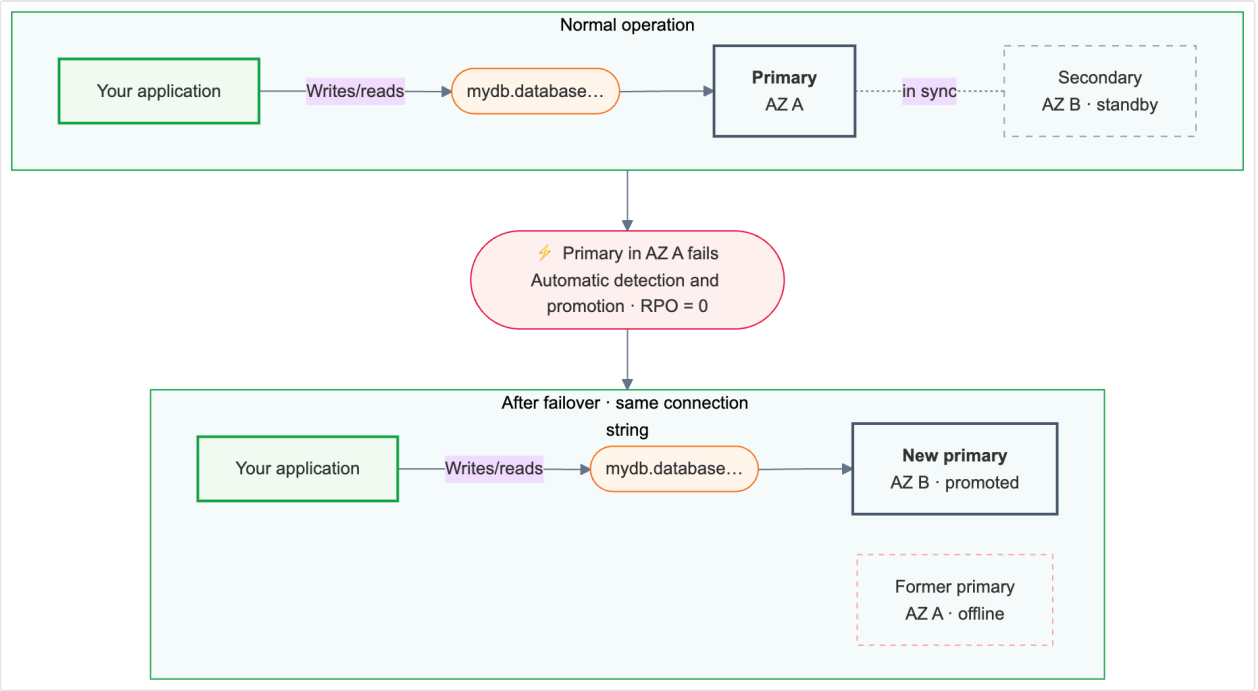
When you need both high availability and additional read capacity beyond what your secondary compute instances provide, you can combine both features on the same branch. See Read replicas.

Failover behavior

Automatic failover

Lakebase monitors primary compute health continuously. If the primary becomes unavailable, failover is triggered automatically.

Failover preserves all committed transactions.



After failover, the primary connection string (`{endpoint-id}.database.{region}.databricks.com`) automatically routes to the newly promoted compute instance. Applications don't need to change their connection configuration, but existing connections are terminated during failover and must reconnect. Applications with retry logic handle this automatically.

Failover with read-only access enabled

When **Allow access to read-only compute instances** is enabled and a failover occurs, the promoted secondary becomes the new primary and stops serving reads. If you have two or more readable secondaries, read traffic on the `-ro` connection string continues at reduced capacity until a replacement is provisioned. If you have only one, reads are fully interrupted until the replacement is ready.

Connection strings

The **Connect** dialog shows both connection strings with their current compute status:

Compute option in Connect dialog	Connection string	Use for
`Primary (name) ● Active`	`{endpoint-id}.database.{region}.databricks.com`	All writes; reads that must hit the current primary
`Secondary (name) ● Active RO`	`{endpoint-id}-ro.database.{region}.databricks.com`	Read offload to secondary compute instances (only available when **Allow access to read-only compute instances** is enabled)

The primary connection string always routes to the current primary, including after a failover.

Each compute instance also has its own direct connection string, accessible from the **Computes** tab via the actions menu (⋮) on each row. Direct connections are intended for troubleshooting individual compute instances, not for application use. Direct connection strings are per-compute and may change when secondaries are added, removed, or promoted.

High availability limits

Limit	Value
Compute instances	2, 3, or 4 (1 primary + 1–3 secondary compute instances)
Autoscaling range (max - min)	≤ 16 CU between minimum and maximum

Limit	Value
Scale to zero	Not available for compute instances in a high availability configuration

Best practices

Following these practices helps your application stay resilient and available during failover events.

Practice	Details
Implement connection retry logic	Active connections are terminated during failover. Connections to the failed primary may hang until timed out — configure TCP keepalives or a connection timeout in your driver to detect failure promptly. Connections to the secondary being promoted are actively terminated, returning an error immediately. Applications with retry logic reconnect automatically within seconds.
Configure secondary count for your use case	Each secondary compute instance represents pre-allocated hardware reserved for failover. Reducing your secondary count means less failover capacity and fewer availability zones covered. One secondary compute instance provides failover coverage. If you enable readable secondaries, configure two or more. With only one, reads are fully interrupted during a failover until a replacement is provisioned.
Avoid overloading secondary compute instances	The service may restart a secondary compute instance that is overloaded or falling behind. Monitor query load and connection counts, and increase CU size if you observe sustained high utilization.

Next steps

- Manage high availability to enable and configure high availability
- Autoscaling for details about CU sizing and autoscaling ranges
- Connection strings for complete connection string reference

Last updated on ***Apr 17, 2026***

Lakebase Data API

> **INFO**
>
> Lakebase Autoscaling is the latest version of Lakebase, with autoscaling compute, scale-to-zero, branching, and instant restore. For supported regions, see Region availability. If you are a Lakebase Provisioned user, see Lakebase Provisioned.

The Lakebase Data API is a PostgREST-compatible RESTful interface that allows you to interact directly with your Lakebase Postgres database using standard HTTP methods. It offers API endpoints derived from your database schema, allowing for secure CRUD (Create, Read, Update, Delete) operations on your data without the need for custom backend development.

Overview

The Data API automatically generates RESTful endpoints based on your database schema. Each table in your database becomes accessible through HTTP requests, enabling you to:

- **Query data** using HTTP GET requests with flexible filtering, sorting, and pagination
- **Insert records** using HTTP POST requests
- **Update records** using HTTP PATCH or PUT requests
- **Delete records** using HTTP DELETE requests
- **Execute functions as RPCs** using HTTP POST requests

This approach eliminates the need to write and maintain custom API code, allowing you to focus on your application logic and database schema.

PostgREST compatibility

The Lakebase Data API is compatible with the PostgREST specification. You can:

- Use existing PostgREST client libraries and tools
- Follow PostgREST conventions for filtering, ordering, and pagination
- Adapt documentation and examples from the PostgREST community

> **NOTE**
>
> The Lakebase Data API is Databricks' implementation designed to be compatible with the PostgREST specification. Because the Data API is an independent implementation, some PostgREST features that aren't applicable to the Lakebase environment aren't included. For details on feature compatibility, see Feature compatibility reference.

For comprehensive details on API features, query parameters, and capabilities, see the PostgREST API reference.

Use cases

The Lakebase Data API is ideal for:

- **Web applications**: Build frontends that directly interact with your database through HTTP requests
- **Microservices**: Create lightweight services that access database resources via REST APIs
- **Serverless architectures**: Integrate with serverless functions and edge computing platforms
- **Mobile applications**: Provide mobile apps with direct database access through a RESTful interface
- **Third-party integrations**: Enable external systems to read and write data securely

Set up the Data API

This section guides you through setting up the Data API, from creating required roles to making your first API request.

Prerequisites

The Data API requires a Lakebase Postgres Autoscaling database project. If you don't have one, see Get started with database projects.

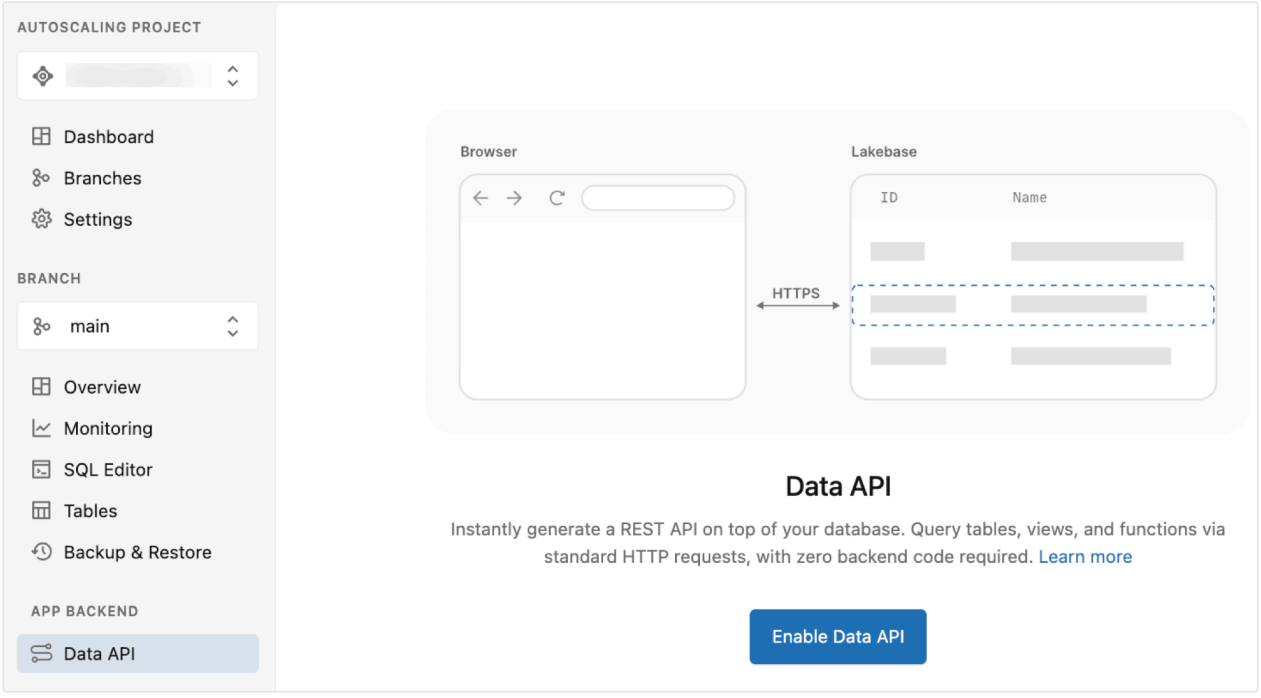
> **TIP**
>
> If you need sample tables for testing the Data API, create them before enabling the Data API. See Sample schema for a complete example schema.

Enable the Data API

The Data API makes all database access through a single Postgres role named `authenticator`, which requires no permissions except to log in. When you enable the Data API through the Lakebase App, this role and the necessary infrastructure are created automatically.

To enable the Data API:

1. Navigate to **Data API** page in your project.
2. Click **Enable Data API**.

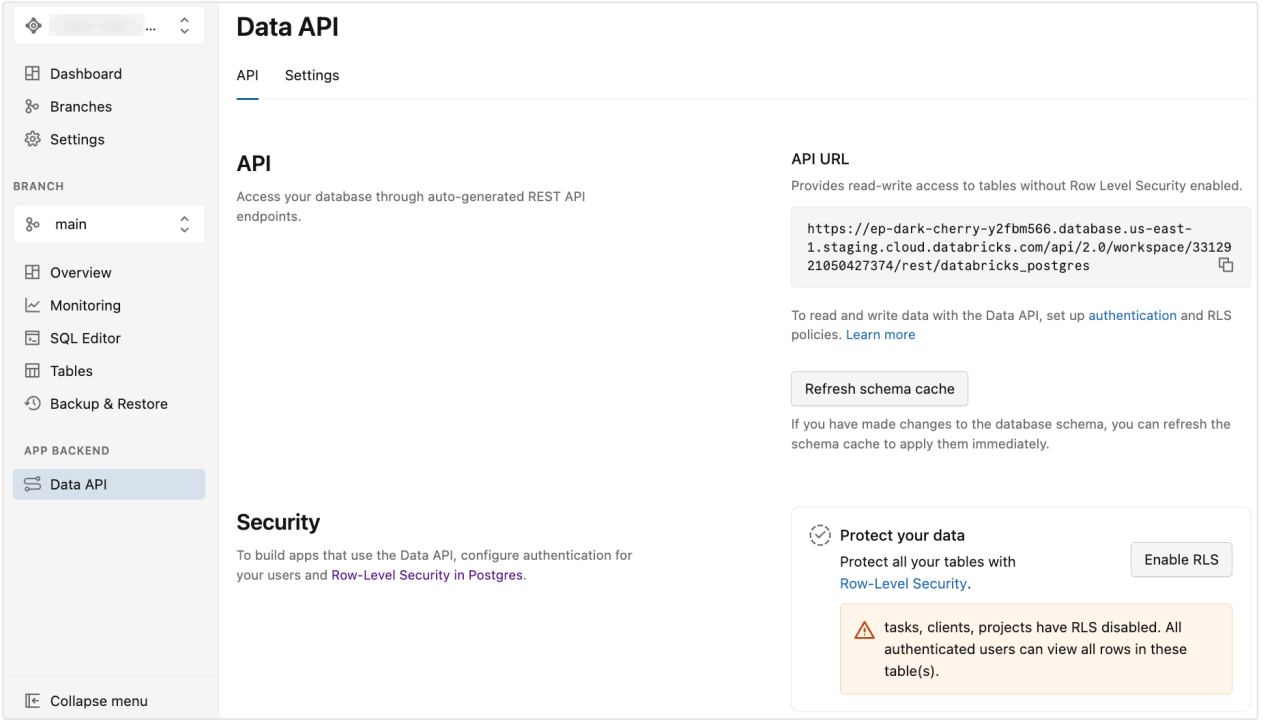


This automatically performs all the setup steps, including creating the `authenticator` role, configuring the `pgrst` schema, and exposing the `public` schema through the API.

> **NOTE**
>
> If you need to expose additional schemas (beyond `public`), you can modify the exposed schemas in the Advanced Data API settings.

After enabling the Data API

After you enable the Data API, the Lakebase App displays the **Data API** page with two tabs: **API** and **Settings**.



The **API** tab provides:

- **API URL**: The REST endpoint URL to use in your application code and API requests. The URL displayed doesn't include the schema, so you must append the schema name (for example, `/public`) to the URL when making API requests.
- **Refresh schema cache**: A button to refresh the API's schema cache after you make changes to your database schema. See Refresh schema cache.
- **Protect your data**: Options to enable Postgres row-level security (RLS) for your tables. See Enable row-level security.

The **Settings** tab provides options to configure API behavior, such as exposed schemas, maximum rows, CORS settings, and more. See Advanced Data API settings.

Sample schema (optional)

The examples in this documentation use the following schema. You can create your own tables or use this sample schema for testing. Run these SQL statements using the Lakebase SQL Editor or any SQL client:

PostgreSQL

```
-- Create clients table
CREATE TABLE clients (
  id SERIAL PRIMARY KEY,
  name TEXT NOT NULL,
  email TEXT UNIQUE NOT NULL,
  company TEXT,
  phone TEXT,
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Create projects table with foreign key to clients
CREATE TABLE projects (
  id SERIAL PRIMARY KEY,
  name TEXT NOT NULL,
  description TEXT,
  client_id INTEGER NOT NULL REFERENCES clients(id) ON DELETE CASCADE,
  status TEXT DEFAULT 'active',
  start_date DATE,
  end_date DATE,
  budget DECIMAL(10,2),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Create tasks table with foreign key to projects
CREATE TABLE tasks (
  id SERIAL PRIMARY KEY,
  title TEXT NOT NULL,
  description TEXT,
  project_id INTEGER NOT NULL REFERENCES projects(id) ON DELETE CASCADE,
  status TEXT DEFAULT 'pending',
  priority TEXT DEFAULT 'medium',
  assigned_to TEXT,
  due_date DATE,
  estimated_hours DECIMAL(5,2),
  actual_hours DECIMAL(5,2),
  created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
);

-- Insert sample data
```

```
INSERT INTO clients (name, email, company, phone) VALUES
  ('Acme Corp', 'contact@acme.com', 'Acme Corporation', '+1-555-0101'),
  ('TechStart Inc', 'hello@techstart.com', 'TechStart Inc', '+1-555-
0102'),
  ('Global Solutions', 'info@globalsolutions.com', 'Global Solutions
Ltd', '+1-555-0103');

INSERT INTO projects (name, description, client_id, status, start_date,
end_date, budget) VALUES
  ('Website Redesign', 'Complete overhaul of company website with modern
design', 1, 'active', '2024-01-15', '2024-06-30', 25000.00),
  ('Mobile App Development', 'iOS and Android app for customer
management', 1, 'planning', '2024-07-01', '2024-12-31', 50000.00),
  ('Database Migration', 'Migrate legacy system to cloud database', 2,
'active', '2024-02-01', '2024-05-31', 15000.00),
  ('API Integration', 'Integrate third-party services with existing
platform', 3, 'completed', '2023-11-01', '2024-01-31', 20000.00);

INSERT INTO tasks (title, description, project_id, status, priority,
assigned_to, due_date, estimated_hours, actual_hours) VALUES
  ('Design Homepage', 'Create wireframes and mockups for homepage', 1,
'in_progress', 'high', 'Sarah Johnson', '2024-03-15', 16.00, 8.00),
  ('Setup Development Environment', 'Configure local development setup',
1, 'completed', 'medium', 'Mike Chen', '2024-02-01', 4.00, 3.50),
  ('Database Schema Design', 'Design new database structure', 3,
'completed', 'high', 'Alex Rodriguez', '2024-02-15', 20.00, 18.00),
  ('API Authentication', 'Implement OAuth2 authentication flow', 4,
'completed', 'high', 'Lisa Wang', '2024-01-15', 12.00, 10.50),
  ('User Testing', 'Conduct usability testing with target users', 1,
'pending', 'medium', 'Sarah Johnson', '2024-04-01', 8.00, NULL),
  ('Performance Optimization', 'Optimize database queries and caching',
3, 'in_progress', 'medium', 'Alex Rodriguez', '2024-04-30', 24.00,
12.00);
```

Configure user permissions

You must authenticate all Data API requests using Databricks OAuth bearer tokens, which are sent via the `Authorization` header. The Data API restricts access to authenticated Databricks identities, with Postgres governing the underlying permissions.

The `authenticator` role assumes the identity of the requesting user when processing API requests. For this to work, each Databricks identity that accesses the Data API must have a corresponding Postgres role in your database. If you need to add users to your Databricks account first, see Add users to your account.

Add Postgres roles

Create a Postgres role for each Databricks identity that needs Data API access:

UI | SQL | Python SDK | curl

1. In **Roles & Databases** > **Add role** > **OAuth** tab, select the user, service principal, or group to grant database access to.
2. After creating the role, proceed to Grant permissions to users.

IMPORTANT

Don't use your database owner account (the Databricks identity who created the Lakebase project) to access the Data API. The `authenticator` role requires the ability to assume your role, and that permission can't be granted for accounts with elevated privileges.

If you attempt to grant the database owner role to `authenticator`, you receive this error:

```
ERROR:  permission denied to grant role
"db_owner_user@databricks.com"
DETAIL:  Only roles with the ADMIN option on role
"db_owner_user@databricks.com" may grant this role.
```

Grant permissions to users

Now that you've created corresponding Postgres roles for your Databricks identities, you need to grant permissions to those Postgres roles. These permissions control which database objects (schemas, tables, sequences, functions) each user can interact with via API requests.

Grant permissions using standard SQL `GRANT` statements. This example uses the `public` schema; if you're exposing a different schema, replace `public` with your schema name:

PostgreSQL | Ask Genie

```
-- Allow authenticator to assume the identity of the user
GRANT "user@databricks.com" TO authenticator;

-- Allow user@databricks.com to access everything in public schema
GRANT USAGE ON SCHEMA public TO "user@databricks.com";
GRANT SELECT, UPDATE, INSERT, DELETE ON ALL TABLES IN SCHEMA public TO
"user@databricks.com";
GRANT USAGE ON ALL SEQUENCES IN SCHEMA public TO "user@databricks.com";
GRANT EXECUTE ON ALL FUNCTIONS IN SCHEMA public TO "user@databricks.com";
```

This example grants full access to the `public` schema for the `user@databricks.com` identity. Replace this with the actual Databricks identity and adjust permissions based on your requirements. For service principals, use the application ID (UUID) as the Postgres role name in `GRANT` statements instead of an email address.

> **IMPORTANT**
>
> **Implement row-level security**: The permissions above grant table-level access, but most API use cases require row-level restrictions. For example, in multi-tenant applications, users should only see their own data or their organization's data. Use PostgreSQL row-level security (RLS) policies to enforce fine-grained access control at the database level. See Implement row-level security.

Authentication

To access the Data API, you must provide a Databricks OAuth token in the `Authorization` header of your HTTP request. The authenticated Databricks identity must have a corresponding Postgres role (created in the previous steps) that defines its database permissions.

Get an OAuth token

Connect to your workspace as the Databricks identity for whom you created a Postgres role in the previous steps and obtain an OAuth token. See Authentication for instructions.

Making a request

With your OAuth token and API URL (available from the **API** tab in the Lakebase App), you can make API requests using curl or any HTTP client. Remember to append the schema name (for example, `/public`) to the API URL. The following examples assume you've exported the `DBX_OAUTH_TOKEN` and `REST_ENDPOINT` environment variables.

Here's an example call with the expected output (using the sample clients/projects/tasks schema):

Bash

```bash
curl -H "Authorization: Bearer $DBX_OAUTH_TOKEN" \
  "$REST_ENDPOINT/public/clients?
select=id,name,projects(id,name)&id=gte.2"
```

Example response:

JSON

```json
[
  { "id": 2, "name": "TechStart Inc", "projects": [{ "id": 3, "name":
"Database Migration" }] },
  { "id": 3, "name": "Global Solutions", "projects": [{ "id": 4, "name":
"API Integration" }] }
]
```

For more examples and detailed information about API operations, see the API reference section. For comprehensive details on query parameters and API capabilities, see the PostgREST API reference. For Lakebase-specific compatibility information, see PostgREST compatibility.

Before using the API extensively, configure Row-level security to protect your data.

Manage the Data API

After enabling the Data API, you can manage schema changes and security settings through the Lakebase App.

Refresh schema cache

When you make changes to your database schema (adding tables, columns, or other schema objects), you need to refresh the schema cache. This makes your changes immediately available through the Data API.

To refresh the schema cache:

1. Navigate to **Data API** in the **App Backend** section of your project.
2. Click **Refresh schema cache**.

The Data API now reflects your latest schema changes.

Enable row-level security

The Lakebase App provides a quick way to enable row-level security (RLS) for tables in your database. When tables exist in your schema, the **API** tab displays a **Protect your data** section that shows:

- Tables with RLS enabled
- Tables with RLS disabled (with warnings)
- An **Enable RLS** button to enable RLS for all tables

> **IMPORTANT**
>
> Enabling RLS through the Lakebase App turns on row-level security for your tables. When RLS is enabled, all rows become inaccessible to users by default (except for table owners, roles with the BYPASSRLS attribute, and superusers—though superusers aren't supported on Lakebase). You must create RLS policies to grant access to specific rows based on your security requirements. See Row-level security for information about creating policies.

To enable RLS for your tables:

1. Navigate to **Data API** in the **App Backend** section of your project.
2. In the **Protect your data** section, review the tables that don't have RLS enabled.
3. Click **Enable RLS** to enable row-level security for all tables.

You can also enable RLS for individual tables using SQL. See Row-level security for details.

Advanced Data API settings

The **Advanced settings** section on the **API** tab in the Lakebase App controls the security, performance, and behavior of your Data API endpoint.

Exposed schemas

Default: `public`

Defines which PostgreSQL schemas are exposed as REST API endpoints. By default, only the `public` schema is accessible. If you use other schemas (for example, `api`, `v1`), select them from the drop-down list to add them.

> **NOTE**
>
> **Permissions apply:** Adding a schema here exposes the endpoints, but the database role used by the API must still have `USAGE` privileges on the schema and `SELECT` privileges on the tables.

Maximum rows

Default: Empty

Enforces a hard limit on the number of rows to return in a single API response. This prevents accidental performance degradation from large queries. Clients should use pagination limits to retrieve data within this threshold. This also prevents unexpected egress costs from large data transfers.

CORS allowed origins

Default: Empty (Allows all origins)

Controls which web domains can fetch data from your API using a browser.

- **Empty:** Allows `*` (any domain). Useful for development.
- **Production:** List your specific domains (for example, `https://myapp.com`) to prevent unauthorized websites from querying your API.

OpenAPI specification

Default: Disabled

Controls whether an auto-generated OpenAPI 3 schema is available at `/openapi.json`. This schema describes your tables, columns, and REST endpoints. When enabled, you can use it to:

- Generate API documentation (Swagger UI, Redoc)
- Build typed client libraries (TypeScript, Python, Go)
- Import your API into Postman
- Integrate with API gateways and other OpenAPI-based tools

Server timing headers

Default: Disabled

When enabled, the Data API includes `Server-Timing` headers in each response. These headers show how long different parts of the request took to process (for example, database execution time and internal processing time). You can use this information to debug slow queries, measure performance, and troubleshoot latency issues in your application.

> **NOTE**
> After making changes to any advanced settings, click **Save** to apply them.

Row-level security

Row-level security (RLS) policies provide fine-grained access control by restricting which rows users can access in a table.

How RLS works with the Data API: When a user makes an API request, the `authenticator` role assumes that user's identity. Any RLS policies defined for that user's role are automatically enforced by PostgreSQL, filtering the data they can access. This happens at the database level, so even if application code tries to query

all rows, the database only returns rows the user is permitted to see. This provides defense-in-depth security without requiring filtering logic in your application code.

Why RLS is critical for APIs: Unlike direct database connections where you control the connection context, HTTP APIs expose your database to multiple users through a single endpoint. Table-level permissions alone mean that if a user can access the `clients` table, they can access **all** client records unless you implement filtering. RLS policies ensure each user automatically sees only their authorized data.

RLS is essential for:

- **Multi-tenant applications**: Isolate data between different customers or organizations
- **User-owned data**: Ensure users only access their own records
- **Team-based access**: Limit visibility to team members or specific groups
- **Compliance requirements**: Enforce data access restrictions at the database level

Enable RLS

You can enable RLS through the Lakebase App or using SQL statements. For instructions on using the Lakebase App, see Enable row-level security.

> **WARNING**
>
> If you have tables without RLS enabled, the **API** tab in the Lakebase App displays a warning that authenticated users can view all rows in those tables. The Data API interacts directly with your Postgres schema, and because the API is accessible over the internet, it's crucial to enforce security at the database level using PostgreSQL row-level security.

To enable RLS using SQL, run the following command:

PostgreSQL

```
ALTER TABLE clients ENABLE ROW LEVEL SECURITY;
```

Create RLS policies

After enabling RLS on a table, you must create policies that define access rules. Without policies, users cannot access any rows (all rows are hidden by default).

How policies work: When RLS is enabled on a table, users can only see rows that match at least one policy. All other rows are filtered out. Table owners, roles with the `BYPASSRLS` attribute, and superusers can bypass the row security system (though superusers aren't supported on Lakebase).

> **NOTE**
>
> In Lakebase, `current_user` returns the authenticated user's email address (for example, `user@databricks.com`). Use this in your RLS policies to identify which user is making the request.

Basic policy syntax:

PostgreSQL

```
CREATE POLICY policy_name ON table_name
  [TO role_name]
  USING (condition);
```

- **policy_name**: A descriptive name for the policy
- **table_name**: The table to apply the policy to
- **TO role_name**: Optional. Specifies the role for this policy. Omit this clause to apply the policy to all roles.
- **USING (condition)**: The condition that determines which rows are visible

RLS tutorial

The following tutorial uses the sample schema from this documentation (clients, projects, tasks tables) to show how to implement row-level security.

Scenario: You have multiple users who should only see their assigned clients and related projects. Restrict access so that:

- `alice@databricks.com` can only view clients with IDs 1 and 2
- `bob@databricks.com` can only view clients with IDs 2 and 3

Step 1: Enable RLS on the clients table

PostgreSQL

```
ALTER TABLE clients ENABLE ROW LEVEL SECURITY;
```

Step 2: Create a policy for Alice

PostgreSQL

```
CREATE POLICY alice_clients ON clients
  TO "alice@databricks.com"
  USING (id IN (1, 2));
```

Step 3: Create a policy for Bob

PostgreSQL

```
CREATE POLICY bob_clients ON clients
  TO "bob@databricks.com"
  USING (id IN (2, 3));
```

Step 4: Test the policies

When Alice makes an API request:

Bash

```
# Alice's token in the Authorization header
curl -H "Authorization: Bearer $ALICE_TOKEN" \
  "$REST_ENDPOINT/public/clients?select=id,name"
```

Response (Alice only sees clients 1 and 2):

JSON

```
[
  { "id": 1, "name": "Acme Corp" },
```

```
  { "id": 2, "name": "TechStart Inc" }
]
```

When Bob makes an API request:

Bash

```
# Bob's token in the Authorization header
curl -H "Authorization: Bearer $BOB_TOKEN" \
  "$REST_ENDPOINT/public/clients?select=id,name"
```

Response (Bob only sees clients 2 and 3):

JSON

```
[
  { "id": 2, "name": "TechStart Inc" },
  { "id": 3, "name": "Global Solutions" }
]
```

Common RLS patterns

These patterns cover typical security requirements for the Data API:

User ownership - Restricts rows to the authenticated user:

PostgreSQL

```
CREATE POLICY user_owned_data ON tasks
  USING (assigned_to = current_user);
```

Tenant isolation - Restricts rows to the user's organization:

PostgreSQL

```
CREATE POLICY tenant_data ON clients
  USING (tenant_id = (
    SELECT tenant_id
    FROM user_tenants
```

```
    WHERE user_email = current_user
  ));
```

Team membership - Restricts rows to the user's teams:

PostgreSQL

```
CREATE POLICY team_projects ON projects
  USING (client_id IN (
    SELECT client_id
    FROM team_clients
    WHERE team_id IN (
      SELECT team_id
      FROM user_teams
      WHERE user_email = current_user
    )
  ));
```

Role-based access - Restricts rows based on role membership:

PostgreSQL

```
CREATE POLICY manager_access ON tasks
  USING (
    status = 'pending' OR
    pg_has_role(current_user, 'managers', 'member')
  );
```

Read-only for specific roles - Different policies for different operations:

PostgreSQL

```
-- Allow all users to read their assigned tasks
CREATE POLICY read_assigned_tasks ON tasks
  FOR SELECT
  USING (assigned_to = current_user);

-- Only managers can update tasks
CREATE POLICY update_tasks ON tasks
  FOR UPDATE
  TO "managers"
  USING (true);
```

Additional resources

For comprehensive information about implementing RLS, including policy types, security best practices, and advanced patterns, see the PostgreSQL Row Security Policies documentation.

For more information about permissions, see Manage permissions.

API reference

This section assumes you've completed the setup steps, configured permissions, and implemented row-level security. The following sections provide reference information for using the Data API, including common operations, advanced features, security considerations, and compatibility details.

Basic operations

Query records

Retrieve records from a table using HTTP GET:

Bash

```bash
curl -H "Authorization: Bearer $DBX_OAUTH_TOKEN" \
  "$REST_ENDPOINT/public/clients"
```

Example response:

JSON

```
[
  { "id": 1, "name": "Acme Corp", "email": "contact@acme.com", "company":
"Acme Corporation", "phone": "+1-555-0101" },
  {
    "id": 2,
    "name": "TechStart Inc",
    "email": "hello@techstart.com",
    "company": "TechStart Inc",
```

```
    "phone": "+1-555-0102"
  }
]
```

Filter results

Use query parameters to filter results. This example retrieves clients with `id` greater than or equal to 2:

Bash

```
curl -H "Authorization: Bearer $DBX_OAUTH_TOKEN" \
  "$REST_ENDPOINT/public/clients?id=gte.2"
```

Example response:

JSON

```
[
  { "id": 2, "name": "TechStart Inc", "email": "hello@techstart.com" },
  { "id": 3, "name": "Global Solutions", "email":
"info@globalsolutions.com" }
]
```

Select specific columns and join tables

Use the `select` parameter to retrieve specific columns and join related tables:

Bash

```
curl -H "Authorization: Bearer $DBX_OAUTH_TOKEN" \
  "$REST_ENDPOINT/public/clients?
select=id,name,projects(id,name)&id=gte.2"
```

Example response:

JSON

```
[
  { "id": 2, "name": "TechStart Inc", "projects": [{ "id": 3, "name":
"Database Migration" }] },
  { "id": 3, "name": "Global Solutions", "projects": [{ "id": 4, "name":
"API Integration" }] }
]
```

Insert records

Create new records using HTTP POST:

Bash

```
curl -X POST \
  -H "Authorization: Bearer $DBX_OAUTH_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{
    "name": "New Client",
    "email": "newclient@example.com",
    "company": "New Company Inc",
    "phone": "+1-555-0104"
  }' \
  "$REST_ENDPOINT/public/clients"
```

Update records

Update existing records using HTTP PATCH:

Bash

```
curl -X PATCH \
  -H "Authorization: Bearer $DBX_OAUTH_TOKEN" \
  -H "Content-Type: application/json" \
  -d '{"phone": "+1-555-0199"}' \
  "$REST_ENDPOINT/public/clients?id=eq.1"
```

Delete records

Delete records using HTTP DELETE:

Bash

```
curl -X DELETE \
  -H "Authorization: Bearer $DBX_OAUTH_TOKEN" \
  "$REST_ENDPOINT/public/clients?id=eq.5"
```

Advanced features

Pagination

Control the number of records returned using the `limit` and `offset` parameters:

Bash

```
curl -H "Authorization: Bearer $DBX_OAUTH_TOKEN" \
  "$REST_ENDPOINT/public/tasks?limit=10&offset=0"
```

Sorting

Sort results using the `order` parameter:

Bash

```
curl -H "Authorization: Bearer $DBX_OAUTH_TOKEN" \
  "$REST_ENDPOINT/public/tasks?order=due_date.desc"
```

Complex filtering

Combine multiple filter conditions:

Bash

```
curl -H "Authorization: Bearer $DBX_OAUTH_TOKEN" \
  "$REST_ENDPOINT/public/tasks?status=eq.in_progress&priority=eq.high"
```

Common filter operators:

- `eq` - equals
- `gte` - greater than or equal
- `lte` - less than or equal
- `neq` - not equal
- `like` - pattern matching
- `in` - matches any value in list

For more information about supported query parameters and API features, see the PostgREST API reference. For Lakebase-specific compatibility information, see PostgREST compatibility.

Feature compatibility reference

The Lakebase Data API is fully compatible with the PostgREST specification, including resource embedding (foreign key and computed relationships, inner/left join hints), response formats (JSON, CSV, GeoJSON, custom media types), filtering, pagination, counting modes (`exact`, `planned`, `estimated`), request preferences (`handling`, `timezone`, `tx`), query plan exposure, and RPC features.

The following PostgREST features are not applicable or not available in the Lakebase environment:

Authentication

Feature	Status	Details
JWT configuration	Not applicable	The Lakebase Data API uses Databricks OAuth tokens instead of JWT authentication. JWT-specific configuration options (custom secrets, RS256 keys, audience validation) are not available.

Advanced configuration

Feature	Status	Details
Application settings (GUCs)	Not supported	Passing custom configuration values to database functions via PostgreSQL GUCs is not supported.
Pre-request function	Not supported	The `db-pre-request` configuration that allows specifying a database function to run before each request is not supported.

Observability

Feature	Status	Details
Trace header propagation	Not applicable	Lakebase implements its own observability features instead of PostgREST's X-Request-Id and custom trace header propagation.

For more information about PostgREST features, see the PostgREST documentation.

Security considerations

The Data API enforces your database's security model at multiple levels:

- **Authentication**: All requests require valid OAuth token authentication
- **Role-based access**: Database-level permissions control which tables and operations users can access
- **Row-level security**: RLS policies enforce fine-grained access control, restricting which specific rows users can see or modify
- **User context**: The API assumes the authenticated user's identity, ensuring database permissions and policies apply correctly

Recommended security practices

For production deployments:

1. **Implement row-level security**: Use RLS policies to restrict data access at the row level. This is especially important for multi-tenant applications and user-owned data. See Row-level security.
2. **Grant minimal permissions**: Only grant the permissions users need (`SELECT`, `INSERT`, `UPDATE`, `DELETE`) on specific tables rather than granting broad access.
3. **Use separate roles per application**: Create dedicated roles for different applications or services rather than sharing a single role.
4. **Audit access regularly**: Review granted permissions and RLS policies periodically to ensure they match your security requirements.

For information about managing roles and permissions, see:

- Manage Postgres roles
- Manage permissions

Last updated on ***Mar 11, 2026***

Lakebase Autoscaling limitations

> **INFO**
>
> Lakebase Autoscaling is the latest version of Lakebase, with autoscaling compute, scale-to-zero, branching, and instant restore. For supported regions, see Region availability. If you are a Lakebase Provisioned user, see Lakebase Provisioned.

The following limitations apply to Lakebase Postgres Autoscaling.

> **NOTE**
>
> This page describes **limitations**, such as features not yet supported and compatibility constraints. If you're looking for **project limits** (for example, maximum number of projects per workspace, branches per project, or history retention), see Project limits in Manage projects.

Migration between Lakebase Provisioned and Lakebase Autoscaling

Direct migration between Lakebase Provisioned and Lakebase Autoscaling is not currently supported.

If you need to move data from a Lakebase Provisioned database to a Lakebase Autoscaling database, you can use pg_dump and pg_restore, which are standard Postgres export and import tools.

Feature availability

This section describes which features and integrations are available in Lakebase Autoscaling and which are currently only available in Lakebase Provisioned.

Compliance

Compliance security profile:

Lakebase Autoscaling supports workspaces with the compliance security profile when the compliance standard is set to HIPAA, C5, TISAX, or None. Other compliance standards are not supported.

For a complete feature comparison between Lakebase Autoscaling and Lakebase Provisioned, see Feature comparison.